

Minimalism and Computational Linguistics

Thomas Graf

1 Introduction

Computational linguistics is a large field that covers a wide range of topics and approaches, from the theory-minded work in mathematical linguistics and formal language theory to very applied issues such as automatic generation of descriptive captions for images. Computational work on Minimalism has largely been theory-focused, but there have been some major advances in the last few years that hold the promise of Minimalist ideas finding their way into more applied domains. This recent work builds on Stabler’s *Minimalist grammars* (MGs) (Stabler 1997, 2011a; see also Kobele, this volume). While MGs are not the only computational formalization of Minimalist syntax (cf. Amblard et al. 2010, Fong and Ginsburg 2012), nor the only synthesis of computational methods and Minimalist ideas (cf. Yang 2002, Frank 2004), they constitute the largest and thematically most varied body of such work to date. Hence this chapter will use MGs as the lens through which the intersection of Minimalism and computational linguistics will be explored.

After a brief introduction to MGs (Sec. 2), I will cover: their weak generative capacity and why it points towards copy-movement as possibly the most important innovation of Minimalism (Sec. 3); how investigations of strong generative capacity have yielded techniques for making MGs highly faithful to Minimalism without losing their core properties (Sec. 4); recent work on subregular complexity that is more sensitive to the empirical phenomena Minimalists care about, and also reveals surprising parallels to phonology (Sec. 5); and finally MGs as a useful tool for incorporating Minimalist ideas into computational modeling and language technology (Sec. 6). Throughout, emphasis will be put on how the computational insights might inform linguistic research, and in what ways linguists might contribute to our computational understanding of language. Either one may take a variety of forms: deeper insights into linguistic machinery, concrete questions about empirical phenomena, unexpected parallels between syntax and phonology or morphology, the joint exploration of sentence processing, or the construction of Minimalist corpora. Still, not every section may be of equal interest to the reader, and hence every section ends with short Cliffs notes that summarize the current lay of the land as succinctly as possible.

In order to keep this chapter accessible to as broad an audience as possible, I will focus on intuitions and the big picture — mathematically inclined readers are referred to the primary literature for the many technical details that are glossed over here. If there is one thing that I hope the reader will get out of this chapter, it is the realization that even if Minimalism and computational linguistics do not march in perfect lockstep, it is now easier than ever for each one to borrow insights from the other in order to unlock new research avenues.

2 Minimalist grammars in a nutshell

True understanding of any formalism involves two components: I) mastery of the technical machinery, and II) the ability to insightfully apply this machinery to concrete problems, e.g. raising, passive, or wh-movement. This chapter is primarily concerned with the former, i.e. the formal nuts and bolts of MGs. For an in-depth demonstration of MGs as a framework of linguistic analysis, the reader should consult Kobele (this volume).

MGs were originally defined in Stabler (1997) as a formalization of Minimalist syntax as presented in Chomsky (1995), although they have developed a lot since then. The goals of formalizing a syntactic theory are manifold. The mathematical rigor makes it possible to address questions such as: How powerful is the formalism, under what specific notion of power? Is it too powerful for natural language, or not powerful enough? What is the source of the formalism's power, and how does it change if certain parameters are altered? What is the formal nature of specific syntactic proposals, and can they be stated in terms that are less theory-laden or dependent on specific notational conventions? Given these insights, which empirical or conceptual issues should linguists pay particular attention to? Are there unexpected predictions that can take syntactic research in completely new directions? The idea is that by gaining a better understanding of syntactic theory, we also get a better understanding of syntax (this will be the focus of Sec. 3, 4, and 5).

But formalization also extends the reach of syntactic theory. Mathematics is a powerful *lingua franca*, a tremendous bridge builder. By putting Minimalism on a mathematical basis, one can link it to existing work on parsing and learnability (see Sec. 6). Not only does this strengthen the connection between theoretical syntax and psycholinguistics, it also unlocks the door to large-scale applications in modern language technology. If Minimalist ideas can be shown to be useful for practical applications, that is mutually beneficial for all involved fields.

The proof of the pudding is in the eating, though, so let us see what the MG pudding has to offer. First we need a definition of MGs. There are many different ones to choose from depending on whether MGs are viewed as generators of phrase structure trees (Stabler 1997), strings (Stabler and Keenan 2003), or a specification of well-formed syntactic derivations (Kobele et al. 2007). These definitions are all formally equivalent; I will present a version here that draws primarily from the

latter two. Every MG will be defined as a finite set of lexical items, from which one then constructs an infinite set of licit syntactic derivations, each one represented as a tree. By assigning specific externalization mechanisms to the Merge and Move steps that take place during the derivations, one can then obtain a set of Bare Phrase Structure trees, X' -trees, multi-dominance trees, PF- or LF-structures, surface strings, and much more. MGs thus describe a three-step pipeline: lexicon (Sec. 2.1), derivation trees (Sec. 2.2), and externalization (Sec. 2.3).

2.1 Features and the lexicon

Following the Chomsky-Borer hypothesis (Borer 1984), MGs situate all language-specific variation in the lexicon. Hence every MG is just a finite set of lexical items. Each lexical item takes the form $A :: \alpha$, where A is the item's phonetic exponent and α its string of features. For example, an MG fragment of English may contain the lexical item $John :: D^-$, indicating that *John* is a DP, or $hit :: D^+.D^+.V^-$, where V^- tells us that *hit* is a verb and $D^+.D^+$ tell us that this verb selects two DPs. Under a VP-shell analysis *hit* would instead correspond to the lexical item $hit :: D^+.V^-$ with only one DP-argument as the subject DP would then be selected by a separate v -head above VP. As explained in Kobele (this volume), VP-shells offer a more succinct description, but in an effort to keep the trees in this chapter as simple as possible, I will omit v and have subjects originate within VP. Whichever analysis one prefers, the choice between them is hashed out in terms of what MG lexicon one posits, i.e. the finite list of pairs of phonetics exponents and feature strings.

We have barely started our discussion of MGs and already there have been several points that may raise eyebrows. First is the nature of the phonetic exponents of lexical items in MGs. Fully inflected phonetic exponents like *did* were commonly assumed in early Minimalism but were jettisoned after the introduction of the Agree operation in Chomsky (1998). Distributed Morphology eschews the notion of phonetic exponents in syntax altogether, replacing it with abstract roots whose phonetic and semantic content is inserted post-syntactically. Nanosyntax pushes this even further by completely dissolving the notion of lexical item and making each feature a leaf in the syntactic structure. In this system, lexical items are but the spell-out of spans of features (see also Svenonius 2020). These are all interesting proposals, and their implications can be studied through the lens of MGs. The way one does that, though, is by first establishing a simple baseline to which the alternative proposals can be compared. As we will see in Sec. 4, the various systems of handling phonetic exponency do not alter generative capacity, although they might affect what is known as subregular complexity (Sec. 5). For now, we will use the standard MG formalism, secure in the knowledge that this won't preclude us from considering variations that have been proposed in the Minimalist literature.

This is perhaps the most important point to understand about MGs: no component or axiom is

perennially set in stone, each one can be swapped out with various alternatives to observe which computational properties of the formalism this affects. The answer to that is often “None that we have the means to measure”. Chomsky (1990:146f) cautions against formalization for formalization’s sake:

To formalize one or another version is a straightforward exercise, but [...] it would require decisions that are arbitrary; not enough is understood to make them on principled grounds. The serious problem is to learn more, not to formalize what is known and make unmotivated moves into the unknown.

MGs sidestep the problem of arbitrary formalization by considering classes of formalizations. Very much like Minimalism is a flexible framework where pieces can be plugged in, removed, and reassembled, MGs aren’t a single formalization but a vast playground of ideas that can be tied together in various ways. The one constant in this deluge of variants and extensions is the standard MG formalism of Stabler (1997); it is the baseline that all other MG incarnations are compared against. Surprisingly often, it turns out that the modifications or extensions under consideration can be translated back to this baseline, and this is why the standard MG formalism, as odd as it may look through Minimalist eyes, has had such staying power.

This class-focused approach also resolves another tension point, which is the MG treatment of the lexicon as a finite list of lexical items, i.e. pairs of phonetic exponents and syntactic features. To call this a simplistic model of the human lexicon is a major understatement. A native speaker’s mental lexicon clearly stores more information for each lexical item than just its pronunciation and syntactic features, at the very least also its semantics. The lexicon arguably isn’t stored as a list, and the lexicon may be infinite given unbounded affixation patterns as in *grandfather*, *great grandfather*, *great great grandfather*, and so on. These points are all correct, but they have no bearing on the MG lexicon. The MG lexicon is not a model of the mental lexicon, it is a specification of what a model of the mental lexicon must provide in order to be compatible with MGs.

As long as it is possible to take a model of the mental lexicon and extract from it a finite list of feature-annotated exponents, this model can be used with MGs. This simple baseline allows for tremendous variation. The mental lexicon is allowed to be infinite as long as this infinity can be reduced to combining a finite number of elements via Merge — the infinity brought about by the *great grandfather* construction above, for example, would be reduced to just two lexical items, one for *grandfather*, and one for *great* as a modifier that can be combined arbitrarily often with *grandfather* as part of MG’s structure-building process. In the other direction, the mental lexicon may well contain supra-lexical material like the intensifier *the (freaking) (flying) fuck*, or entire proverbs such as *what goes around comes around*. This can still be reduced to a finite list of individual lexical items, some of which just happen to have a specific meaning when they

appear in a structural configuration that matches the entries in the mental lexicon. Quite generally, one can posit all kinds of additional information for each lexical item, e.g. lambda formulas or high-dimensional vectors for semantic interpretation, or lexical frequency information for parsing. One can also assume completely different kinds of lexical data structures, e.g. prefix trees, finite-state automata, or distributed subsymbolic representations as in neural networks; any mental data structure is fine as long as it can be converted to a finite list of lexical items. Very much like Minimalist syntax, MGs are completely agnostic about the nature of the mental lexicon and merely presuppose the existence of some storage of finitely many syntactic atoms.

Let us consider in detail, then, what the lexicon of a standard MG looks like. Below is a small toy grammar for English, with each lexical item a pair of a fully inflected phonetic exponent and a finite, non-empty string of features.

(1) **A small MG fragment for English**

car :: N ⁻	the :: N ⁺ .D ⁻	hit :: D ⁺ .D ⁺ .V ⁻
John :: D ⁻	the :: N ⁺ .D ⁻ .nom ⁻	ε :: V ⁺ .nom ⁺ .T ⁻
John :: D ⁻ .nom ⁻	which :: N ⁺ .D ⁻ .wh ⁻	ε :: T ⁺ .C ⁻
	which :: N ⁺ .D ⁻ .nom ⁻ .wh ⁻	did :: T ⁺ .wh ⁺ .C ⁻

Each feature string encodes a lot of information: the order in which features must be checked, whether they are Merge features (in upper case) or Move features (in lower case), and what their polarity is (which is crucial for feature checking).

Features must be checked in the order in which they appear on the lexical item. For example, *doubt* :: C⁺.D⁺.V⁻ must take a CP as its first argument, followed by a subject DP as its second argument, before it can be selected by another head that is looking for a VP. While certainly not a common assumption among Minimalists, ordered features are close in spirit to Minimalist proposals that use a very explicit feature calculus, e.g. Adger (2003) and Müller (2010). Feature checking can only occur between matching features of opposite polarity. Hence N⁺ and N⁻ can check each other, but N⁺ and D⁻ cannot, nor can N⁻ and N⁻. Each feature checking step deletes the involved features and then triggers a specific operation depending on the type of the deleted features. Uppercase features like N⁺ and D⁺ trigger *Merge*, whereas lowercase features like wh⁺ and nom⁻ trigger *Move*. With two types (*Merge*, *Move*) and two polarities (+, -), there are four different kinds of MG features.

(2) **Names of MG feature types¹**

¹In the MG literature, one often finds a different feature notation. A category feature is written as *f*, the corresponding selector feature as = *f*, the licenser feature as +*f*, and the licensee feature as -*f*. This removes the need for capitalization to distinguish Merge and Move features, and it makes it easy to define additional variants like ≈ *f* for adjunction, or *f* = for a selector feature that always linearizes the argument to the right of the head. But this notation obscures that selector and licenser features form a natural class by virtue of being positive features, and similarly for

	<i>Merge</i>	<i>Move</i>
+	selector feature F^+	licensor feature f^+ (occurs at target site)
–	category feature F^-	licensee feature f^- (occurs on mover)

MG polarities are quite different from the feature system of interpretable and uninterpretable features in Chomsky (1995), the subsequent switch to valued and unvalued features (Chomsky 2001), or any hybrid of the two (Pesetsky and Torrego 2007). Moreover, few Minimalists assume such a strict split between Merge features and Move features. However, these deviations from Minimalist syntax are less significant than they may seem. The MG feature calculus is not an ontological claim about how actual syntactic features operate, just as its definition of a lexicon as a finite set of feature-annotated lexical items is no theory of the mental lexicon. Both are just convenient tools to regulate the application of Merge and Move, which is the actual core of MGs. MG features could have been defined differently to match syntactic ideas more closely; and as will be explained in Sec. 4, features could even be done away with completely to allow for free Merge and Move in the vein of Chomsky (2013). But to reiterate a point I made earlier: the current system is mathematically convenient and fertile, and the MG practice is to start with a simple system first and then adapt and expand it as one develops a deeper understanding of its formal plumbing.

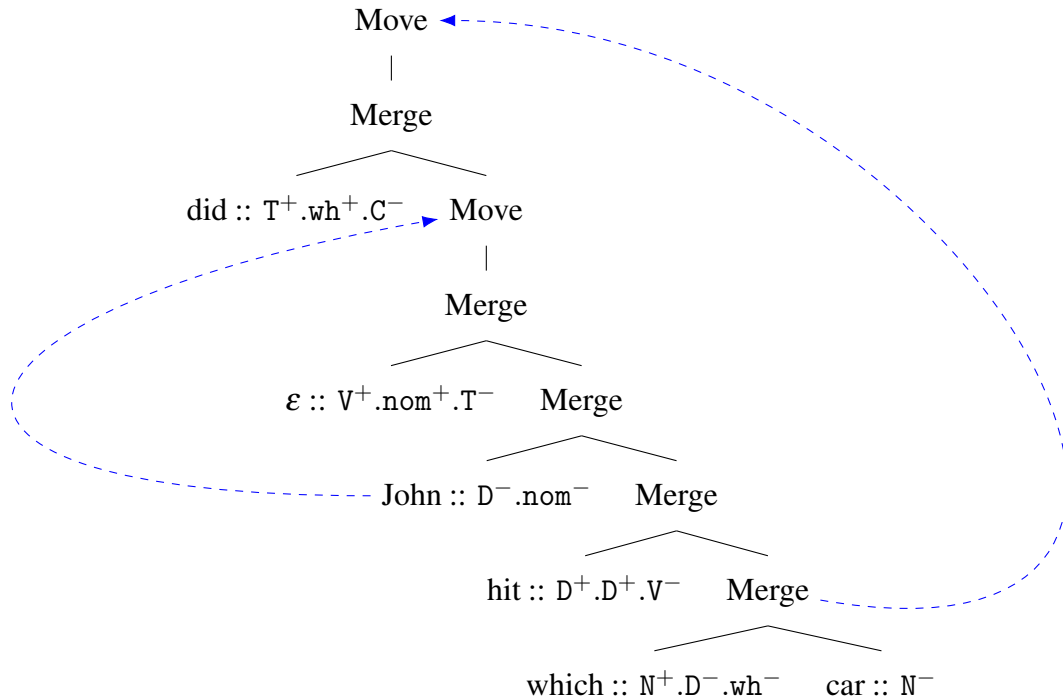
2.2 Derivations as trees

Given some fixed MG G , i.e. some finite set of feature-annotated lexical items, one can look at how the lexical items of G may be combined via Merge and Move as part of a syntactic derivation. Example (3) uses a tree representation of a specific MG derivation that yields *which car did John hit*, with wh-movement of the object to Spec,CP and the subject starting out as an argument of the verb before moving to Spec,TP.² MG derivation trees like this are read bottom-up, with lexical items as the leaves and each interior node denoting a specific operation that takes place at this point of the syntactic operation. It is common to also add movement arrows, but these are not part of the derivation tree and only serve as a visual aid.

(3) MG derivation tree for *which car did John hit*

category and licensee features as negative features. The split between positive and negative features matters at various points in the formalism, e.g. for label projection, and the notation used in this chapter brings out this contrast very clearly.

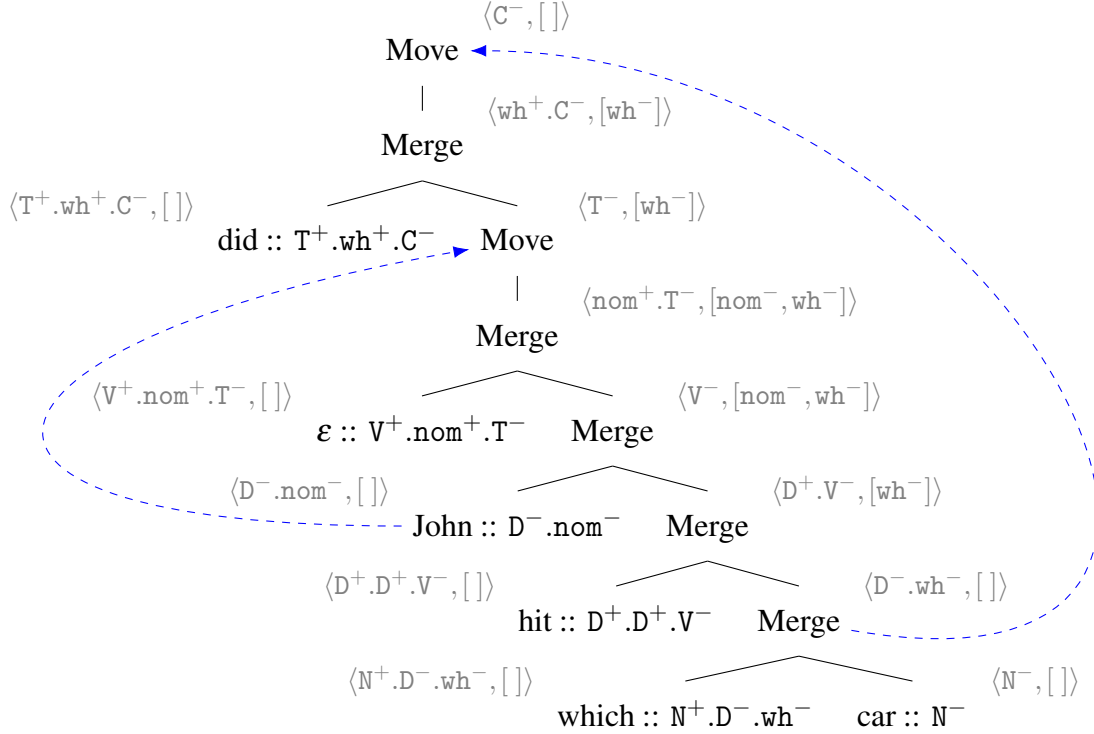
²Throughout this paper, I use the feature *nom* for subject movement. This should not be construed as endorsing the view that subject movement is triggered by the need to check a nominative case feature. The feature name is purely meant as a mnemonic, and one could just as well call the feature *epp*, *occ* (for *occurrence*), or *sub* (for *subject*). However, each one of those names has its own drawbacks — *epp* could be taken to endorse the EPP as a syntactic principle, *occ* might cause confusion with a very different use of the term *occurrence* in the MG literature, and *sub* is at odds with the core Minimalist tenet that subjecthood is not a syntactic primitive. Among many suboptimal naming choices, *nom* has the advantage of being both easy to remember and also too simplistic to be mistaken for a serious theoretical proposal.



Note that movement does not actually displace any subtrees in the derivation tree. The derivation tree is merely a record of the (partial) order in which lexical items enter the derivation and are combined via Merge and Move. As movement does not change the point when an item entered the derivation, there is no displacement in the derivation tree, just an indication that the Move operation is taking place. What is perhaps more confusing is that the Move operation seems to take a single structure as its input, whereas Merge takes two. If we want to fully appreciate this arity difference between Merge and Move, we have to dive deeper into the MG feature calculus.

In order to make fully explicit the MG feature calculus that produces this derivation, I annotate each node with a pair where the first component is the remaining feature string of the current head while the second component is a list consisting of the unchecked parts of all other feature strings that still occur in the whole subtree. I will call such pairs **f[eature]-expressions**.

(4)



For example, the empty T-head $\varepsilon :: V^+.nom^+.T^-$ is annotated with the f-expression $\langle V^+.nom^+.T^-, [] \rangle$ because all the features of the T-head are still active, and the subtree contains no other strings of unchecked features. Its sister Merge node, on the other hand, has the f-expression $\langle V^-, [nom^-, wh^-] \rangle$ because the V-head still need to get its category feature V^- checked, and the subtree contains two incompletely checked feature strings: *which* :: $N^+.D^-.wh^-$ still needs to get its licensee feature wh^- checked, and *John* :: $D^-.nom^-$ has an unchecked licensee feature nom^- . Once the T-head's selector feature V^+ is checked against the V-head's category feature V^- , triggering Merge in the process, we are left with the unchecked features $nom^+.T^-$ on the T-head as well as the unchecked licensee features on *which* and *John*, and thus the corresponding f-expression is $\langle nom^+.T^-, [nom^-, wh^-] \rangle$. At this point, the first unchecked feature of the T-head is nom^+ , which can be checked against the remaining nom^- on *John* to trigger movement, yielding the new f-expression $\langle T^-, [wh^-] \rangle$.

We can now solve the puzzle of Move as a unary operation. As can be seen in this example, Merge and Move differ not only in the types of features involved (selector and category features for Merge, licenser and licensee features for Move). The two operations also differ in where these features come from. Merge always combines two separate f-expressions into a new f-expression. Move, on the other hand, applies to a single f-expression that starts with a licenser feature f^+ and then reaches inside the list of that f-expression to find a feature string that starts with a matching licensee feature f^- — in MGs, Move is literally internal Merge.

This annotation system also makes it easy to state when an MG derivation is well-formed. The derivation only needs to meet two conditions: First, the root node must be annotated with $\langle C^-, [] \rangle$,

which ensures that the derivation yields a CP with all features checked except the category feature on the head. Second, at no point in the derivation may there be an f-expression whose list contains two feature strings that start with the same feature. This is known as the **Shortest Move Constraint** (SMC) but has little to do with the eponymous constraint in Chomsky (1995).

The SMC is at odds with many Minimalist analyses. For example, a natural MG implementation of the standard analysis for sentences like *who likes what* would posit that both wh-phrases carry the licensee feature wh^- but only the subject gets to move because it is closer to the targeted Spec,CP position. But under this proposal the VP would correspond to the f-expression $\langle V^-, [wh^-, wh^-] \rangle$, with both the subject and the object carrying an unchecked wh^- . This f-expression violates the SMC. Why, then, do standard MGs contain such a strong constraint? There are two equally valid answers: first, MG features have a subtly different interpretation from Minimalist features, and second, the SMC is a computational safety measure.

Let us consider the role of features first. In Minimalist syntax, features encode what we may call a lexical item's *syntactic potential*. For example, a wh-feature on *which* encodes that (the phrase headed by) *which* is a valid candidate for wh-movement, but other factors may cause this potential not to be realized, e.g. locality constraints. MG features, on the other hand, encode a lexical item's *syntactic duty*. The lexical item *which* :: $N^+.D^-.wh^-$ absolutely must undergo wh-movement. That this does not always happen is captured by the fact that there is also a lexical item *which* :: $N^+.D^-$ without wh^- . In other words, MGs express syntactic potential indirectly by providing multiple versions of the same lexical item, one with a movement feature, the other without it. MGs make syntactic duty the basic property and syntactic potential an indirect result of lexical variation, whereas Minimalist syntax starts out with syntactic potential and then determines syntactic duty from the structural context. In the proposed MG derivation of *who likes what* with both *who* :: $N^+.D^-.wh^-$ and *what* :: $N^+.D^-.wh^-$, both wh-phrases have a syntactic duty to wh-move, which does not match the spirit of the Minimalist analysis. Actually, then, it is desirable that the SMC rules this out (multiple wh-movement will be discussed in Sec. 4 as the one case where this is still undesirable).

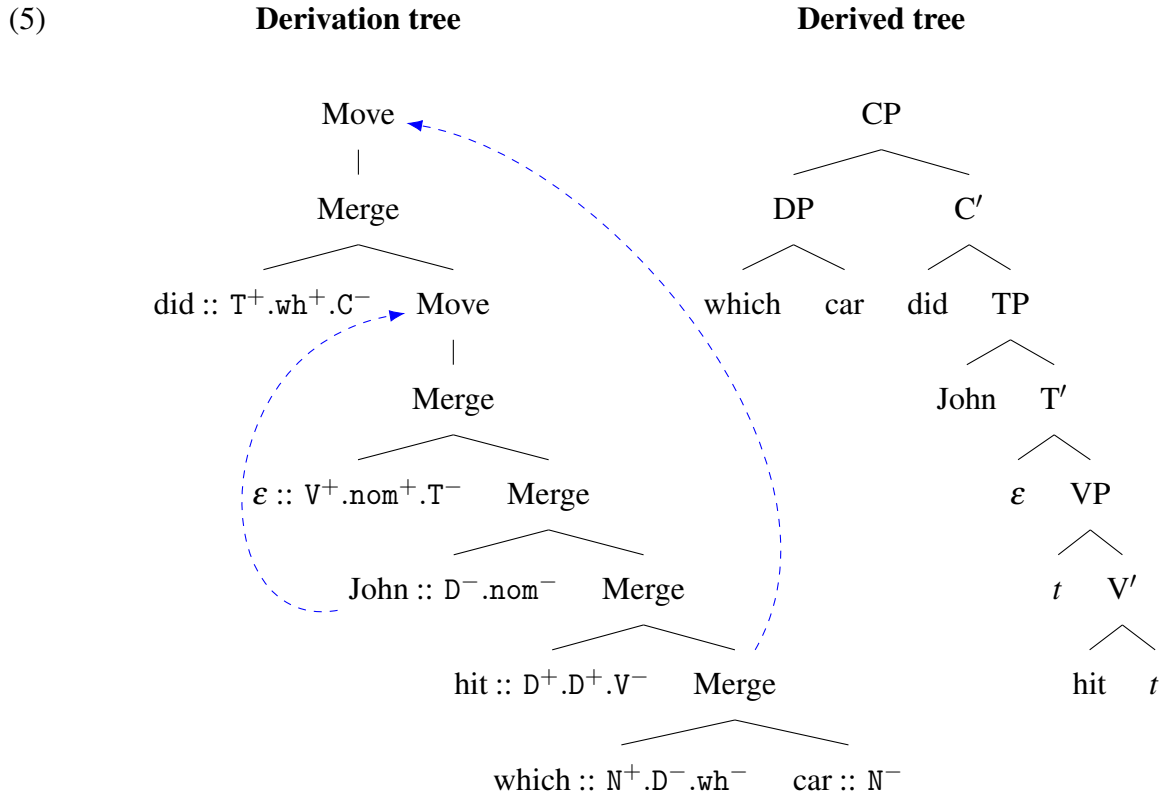
The second point in favor of the SMC is a mathematical one. The SMC serves two technical purposes. First, it keeps the expressive power of MGs in check (see Sec. 3.2). Second, it ensures that Move is a deterministic operation — whenever some licensor feature f^+ gets checked, there is no ambiguity as to which f^- feature is involved in this checking operation (this is also why the movement arrows in (3) are just an optional visual aid). As we will see in Sec. 4, there are now more elegant means of guaranteeing those two properties, so that the SMC is no longer required. However, MGs with the SMC are still easier to study mathematically, and this is why they are the default.

The simple derivational calculus outlined so far is the core of MGs. For each MG G , the feature

calculus determines all the syntactic derivations that are well-formed according to the feature-annotated lexical items provided by G . We can represent each derivation as a tree with lexical items as leaves, Move steps as unary branching nodes, and Merge steps as binary branching nodes. A derivation is well-formed iff I) it obeys the SMC (no node in the derivation corresponds to an f-expression of the form $\langle \alpha, [\dots, f^- \beta, \dots, f^- \gamma, \dots] \rangle$) and II) the only unchecked feature is C^- on the lexical item that heads the whole sentence (or equivalently, the derivation tree's root node corresponds to the f-expression $\langle C^-, [] \rangle$).

2.3 Externalization

Every syntactic derivation can be deterministically translated into the syntactic representation it computes, e.g. a phrase structure tree with traces. Formally, we can think of this as a process that directly transforms a derivation tree into its corresponding output. For phrase structure trees with traces, this involves little more than relabeling interior nodes and placing each mover in its final landing site as the derivation tree already closely mirrors the structure of the phrase structure tree.



Relabeling pays close attention to the polarity of features: if an interior node corresponds to an operation that checks a positive feature on a head h , then this node will be treated as a projection of

h. The lower Move node in (5), for example, is translated to TP because I) this application of Move checks nom^+ on the T-head, and II) the label must be a projection of the T-head because nom^+ is a positive feature, and III) nom^+ is the last positive feature on the T-head, which means that the label cannot be T' , leaving TP as the only option. The Merge node below this lower Move node, on the other hand, yields the label T' as this Merge step checks V^+ , which is a positive feature of the T-head but not the last such feature. Relabeling provides a clear illustration of selector features and licenser features forming a natural class to the exclusion of category and licensee features, which is why it is helpful to think of the former as having opposite polarity from the latter. With feature polarities, it is very easy to keep track of label projection to construct the phrase structure tree computed by a given syntactic derivation.

Once the phrase structure tree has been constructed, its string yield can be obtained in the usual manner. Alternatively, there are also ways to directly compute the string yield from the derivation tree (cf. Stabler and Keenan 2003). We could have also built other kinds of structures, e.g. X' trees, multi-dominance trees, prosodic trees as presented in Richards (2016), and much more. In all cases, the derivation tree remains the same, it is only the externalization of Merge and Move that is altered to compute different outputs.

In contrast to the MG feature calculus and the SMC, the conversion from a derivation tree to the output structure of choice contains barely any surprises for anybody accustomed to Minimalist syntax. Again some technical notions such as Transfer (Chomsky 2008) are not explicitly incorporated, but this is in line with standard mathematical inquiry: we want to understand the simple before we make it complex. Recasting the externalization process as a transformation of the derivation tree into the desired output structure is mathematically simple while still providing a handle on more complex cases, e.g. suppletion and idioms, which are the result of translating a whole chunk of derivational structure into a single node in the PF or LF structure, respectively (Kobele 2012). For example, the compositional derivational structure for *cause to die* may be flattened to just *kill* in the PF-translation, and *kick the bucket to die* at LF. MGs thus offer a purely compositional approach to syntactic derivations while still allowing for non-compositionality in the output structures.

2.4 Cliffs notes

To sum up, MGs mirror Minimalism in that they provide a very flexible framework within which numerous ideas can be fruitfully explored. The baseline that all MG work builds on is defined in Stabler (1997) and more recently Stabler (2011a). These standard MGs consists of a finite set of lexical items, where each lexical item consists of a fully inflected phonetic exponent and a finite string of features. There is an explicit split between Merge features and Move features,

and each feature has a polarity (+ or –). Feature checking is allowed only between features that have opposite polarity but differ in no other respect. The order in which features of a lexical item are checked must match their order in the feature string. Feature checking triggers Merge or Move depending on the type of features involved. The precise effects of Merge and Move depend on what output structure is being assembled. A derivation is well-formed iff I) the only unchecked feature is the category feature C^- on the lexical item that heads the whole tree, and II) the derivation obeys the SMC: at no point in the derivation are there two distinct lexical items that have the same licensee feature as their first unchecked feature. Virtually all these aspects of MGs have been altered at some point to bring them more in line with Minimalist syntax or to explore some computational issues. But rarely does this result in a formalism that cannot be translated back to standard MGs, which is why this particularly frugal version of MGs has stood the test of time.

3 Linguistic insights from weak generative capacity

Defining a computational formalism is of little interest on its own, it is the study of its properties that breathes life into the formalism. Keeping in the spirit of starting with simple concepts that more complex ones can be built on, one should first consider **weak generative capacity**: I) how complex are the string sets that the formalism can generate, II) how complex are natural languages if we view them as sets of well-formed strings, III) how close do specific formalisms get towards characterizing the class of natural languages in this case, and IV) what does this tell us about language and linguistic theories?

Those are very reductionist questions. Section 2 explicitly introduces MGs as generators of syntactic derivations, not strings, so it might seem misguided to focus on the latter rather than the former. Even more obviously, no natural language is adequately described as a collections of well-formed word sequences. Linguists have pointed this out many times, and it is one of the reasons why weak generative capacity no longer receives the attention among generativists that it enjoyed in the early days of Transformational Grammar (Chomsky 1956, 1959; see also Pullum and Gazdar 1982 and references therein). Some might also argue that given the split between I-language and E-language (Chomsky 1986), weak generative capacity cannot enjoy any kind of ontological or cognitive reality and is hence anathema to linguistic concerns. While that may well be so, and while weak generative capacity is the proverbial sledgehammer in the toolbox of formal language theory, it has proven very useful. This is perhaps the most interesting aspect of weak generative capacity: although it is possible to construct artificial scenarios where weak generative capacity tells us very little about language and grammar formalisms, this does not seem to be the case in practice.

In the following, I first address some general issues surrounding weak generative capacity

(Sec. 3.1) before discussing specific results about MGs that are of interest to Minimalist syntax (Sec. 3.2), in particular: the importance of remnant movement and copy movement; insights from Chomsky (1957) and Chomsky (1965) about the succinctness of movement that are corroborated by MGs nowadays; and how the expressivity of MGs is (not) affected by various extensions such as adjunction, sideways movement, and Late Merge.

3.1 Is weak generative capacity relevant?

Since weak generative capacity is a contentious concept among many linguists, its inclusion here requires motivation. What, specifically, does weak generative capacity have to offer to computational and/or theoretical linguistics? Among other things, it provides a fairly theory-neutral classification of natural language phenomena, which in turn provides a lower bound on what grammar formalism have to be capable of.

The best-known example of that is the demise of GPSG (Gazdar et al. 1985). GPSG was built on the assumption that a *context-free* formalism is sufficient to handle all syntactic phenomena, where context-free refers to a specific class of string languages (cf. Fig. 19.1). But Huybregts (1984) and Shieber (1985) provided a powerful argument against that in the form of crossing dependencies that arise with embedding constructions in Dutch and Swiss German, respectively. This argument consists of three steps: First, embedding constructions in Dutch and Swiss German can yield strings that contain patterns of the form $NP_1 NP_2 \cdots NP_n V_1 V_2 \cdots V_n$, where the i -th NP is an argument of the i -th verb. Next, the competence-performance distinction entails that one ought not posit a finite cutoff point k in the grammar such that this construction does not work once n exceeds k . Finally, a series of mathematical steps is used to convert the natural language data sample into a formal language that is provably not context-free.³ As a result, Dutch and Swiss

³The specific series of mathematical steps has to be carefully chosen to avoid what is known as the *subset fallacy* (Mohri and Sproat 2006). In order to show that the complexity of a given (string or tree) language L is at least C , it is not sufficient to identify a subset of L of complexity C or more. To wit, the simplest formal language is the set Σ^* of all possible strings over a fixed alphabet Σ . This language is maximally simple because it exhibits no constraints on strings at all, literally any combination of symbols drawn from Σ is well-formed. Yet Σ^* contains many subsets that are so complex that they aren't even computable. In fact, there are more such uncomputable subsets than there are computable ones.

This scenario arises because complexity is not monotonic. Adding new strings to a formal language can actually reduce its complexity. As a concrete example, consider the language of all strings that contain an odd number of *as*. Verifying for any given string whether it belongs to this language is challenging for a human. We have to carefully keep track of all *as*, no matter where they are in the string, and check that their number is odd. But now suppose that we enrich this language by adding all strings that contain a positive, even number of *as* (i.e. 2, 4, 6, and so on). Now we are dealing with a much simpler language: every string contains an odd number of *as*, or an even number of *as* that is at least 2 — in other words, every string contains at least one *a*. This condition is much easier to verify, we just scan the string from left to right until we find our first *a*. Instead of considering all *as* in the string, we only need to determine whether there is a first one.

The same can happen in the study of natural language through a formal lens. By focusing on a particular construction in isolation (“have an odd number of *as*”), we may miss the fact that there are additional factors (“you can also have an

German are at least **mildly context-sensitive** (Joshi 1985) and cannot be adequately described by a context-free formalism (see also Maclachlan and Rambow 2002 for a more recent argument that makes the same point based on data from Tagalog). These weak generative capacity results show that the context-free GPSG formalism necessarily undergenerates, which renders it inadequate as a model of syntax.

GPSG practitioners were thus left with three choices. They could reject the competence-performance distinction and assume some finite cutoff point for crossing dependencies. But this would have been unsatisfying as GPSG was built on the assumption that other embedding constructions are unbounded. Alternatively, they could have doubted the validity of the data presented by Huybregts and Shieber, or at least how the finite data sample was generalized to an infinite language — perhaps a different formal language would have been context-free while accommodating the data. This was indeed attempted by Pullum and Gazdar (1982) in response to the Dutch data, but their solution failed to extend to Swiss German. The only remaining option is the one that was actually taken in the end: the formalism was modified to handle the new data, which led to the creation of HPSG (Pollard and Sag 1994). This shows that weak generative capacity can be a powerful factor in deciding the linguistic adequacy of a formalism and can drive new developments in syntax.

Crucially, many of the standard responses in syntactic debates were not available to GPSG practitioners. They could not disagree with the theoretical priors because the central empirical claim is purely descriptive. They could not refute the assumed structures because no structure is assumed beyond the string level. They could not reasonably cast doubt on the presumed syntactic dependencies because the only relevant dependency is argument-hood, which is obvious through the semantics and is also indicated via case marking in Swiss German. This reliance on very few, largely innocuous assumptions is the strength of weak generative capacity arguments. Weak generative capacity arguments lack syntactic sophistication, but this is exactly what gives them tremendous empirical robustness — it is a feature, not a bug.

3.2 Minimalism and weak generative capacity

Weak generative capacity also provides some key insights into Minimalism via the proxy of MGs. It was quickly realized that MGs can handle unbounded crossing dependencies and thus are not undermined by the Dutch and Swiss German data. More precisely, Harkema (2001a) and Michaelis (2001) showed that MGs generate **multiple context-free languages** (MCFLs).

This result on its own means precious little without the broader context from formal language

even number of *as* as long as it is at least 2”) so that the language as a whole is actually very simple. Huybregts (1984) and Shieber (1985) use specific techniques to avoid this issue, but as Pullum and Gazdar (1982) discuss at length, a lot of earlier work on the string complexity of natural language fell prey to the subset fallacy.

theory. Formal language theory classifies sets of strings according to their complexity, where complexity can be measured in various ways. Automata theory, for example, measures complexity in terms of the inference rules and memory architectures that are needed for certain kinds of dependencies. Alternatively, one could look at how powerful a mathematical logic is needed to describe these dependencies, or one could express the dependencies as algebraic equations and then measure the complexity of these algebras. There are numerous ways of establishing complexity differences between formal languages, but it is a purely mathematical enterprise the details of which need not concern us here. What is important is the general lay of the land that emerges from this enterprise.

String languages can be put into complexity classes, and those classes can be arranged in a hierarchy of increasing power. Whenever we have $A < B$ in that hierarchy, that means languages in class B can exhibit all dependencies we can find in languages of class A , plus some new kinds of dependencies. Figure 19.1 gives a refined version of the Chomsky-hierarchy, which is the most commonly used classification scheme (but far from the only one). The hierarchy starts with the class REG of regular languages and then goes in increasing order of complexity until we hit the class RE of recursively enumerable languages. RE is the highest level of complexity and includes dependencies that go far beyond anything we find in natural languages — for example, RE includes the language of all strings whose length is a prime number. Some linguistic formalisms, however, have the ability to generate every language that belongs to RE. This includes HPSG and (a specific version of) the original Aspects model of Chomsky (1965).⁴ While RE is too powerful a class, REG is not powerful enough, and language arguably occupies some middle ground between the two extremes.

REG already subsumes almost all dependencies we find in phonology, including local dependencies like word-final devoicing and intervocalic voicing, iterated local dependencies in vowel harmony, and unbounded dependencies as in certain kinds of sibilant harmony (see Heinz 2018 for a full overview). But it crucially lacks nested dependencies, which arise in syntax with center embedding, and crossing dependencies, which Huybregts (1984) and Shieber (1985) found in Dutch and Swiss German, respectively. Center embedding falls within the class of context-free languages (CFLs), and linguistic formalisms at this level of expressivity include GPSG and the original formulation of Categorical Grammar (CG). Crossing dependencies, on the other hand, span a wider range of classes. The dependencies found in Dutch and Swiss German, but also in total reduplication, belong to the class of Tree Adjoining languages (TAL), named like that because they were originally defined as the string languages generated by Tree-Adjoining grammars (TAG). This, however, still does not include all patterns found in language. Total triPLICATION, for example,

⁴The widely recognized result was established in Peters and Ritchie (1971, 1973b). What is not nearly as well known is that Peters and Ritchie (1973a) showed later on that some versions of the Aspects model have more limited expressivity, although they can still generate some recursively enumerable languages.

	mildly context-sensitive										
Class	REG	<	CFL	<	TAL	<	MCFL	<	PMCFL	...	RE
	regular		context-free		tree-adjoining		multiple context-free		parallel MCFL		recursively enumerable
Formalism	SPE		GPSG CG		TAG CCG		MGs with standard movement		MGs with copy movement		Aspects model, HPSG
Example string set	set of all even-length strings		set of all palindromes		ww ($w = \text{any string}$)		www ($w = \text{any string}$)		a^{2^n}		all theorems of first-order logic
Natural phenomena	most of phonology and morphology		center embedding		crossing dependencies, reduplication		total triplication		relativized predicates in Yoruba, case stacking in Old Georgian		

Figure 19.1: Classes of string languages that play a key role in computational linguistics, sorted by complexity; each class is given with at least one formalism that generates all and only those string languages that belong to the class, as well as example string sets and related natural language phenomena (compiled from Peters and Ritchie 1973b, Culy 1985, Joshi 1985, Kaplan and Kay 1994, Vijay-Shanker and Weir 1994, Michaelis and Kracht 1997, Harkema 2001a, Michaelis 2001, Kobele 2006)

requires one step up to MCFLs. Beyond the level of MCFLs are many other classes, including the context-sensitive and recursive languages, until we finally reach RE. With all this information, we can now appreciate what it means that MGs are at the complexity level of MCFLs: they are capable of generating a wide range of phenomena that arise in syntax, including various nested and crossing dependencies; they are more powerful than competing formalisms like TAG and CCG; but they are also far away from the massive overgeneration of the HPSG formalism and the Aspects model.

If we stopped at this general insight about the expressivity of MGs, that would be a mildly convincing argument in their favor as a model of syntax, but it would also sell short just how much the proofs of Harkema (2001a) and Michaelis (2001) actually tell us about the formalism. With weak generative capacity results, the true insights are often in the proofs, not the theorems. The proof that MGs can generate all MCFLs, and nothing more complex than that, is mathematically challenging, but the underlying intuition is simple enough that we may dare take a closer look.

Consider once more the derivation tree in (4), annotated with f-expressions. We thought of this in a bottom-up fashion such that one or two f-expressions are combined into a new f-expression via feature checking. For example, $\langle T^+, wh^+.C^-, [] \rangle$ and $\langle T^-, [wh^-] \rangle$ combine into the new f-expression $\langle wh^+.C^-, [wh^-] \rangle$. But we could also look at it in a top-down fashion: $\langle wh^+.C^-, [wh^-] \rangle$ is split or

unmerged into $\langle T^+.wh^+.C^-, [] \rangle$ and $\langle T^-, [wh^-] \rangle$. Similarly, $\langle T^-, [wh^-] \rangle$ can be *unmoved* to yield $\langle nom^+.T^-, [nom^-, wh^-] \rangle$. Both examples are easily represented as context-free rewrite rules:

(6) **Unmerge and Unmove as rewrite rules**

$$\begin{aligned} \langle wh^+.C^-, [wh^-] \rangle &\rightarrow \langle T^+.wh^+.C^-, [] \rangle \langle T^-, [wh^-] \rangle \\ \langle T^-, [wh^-] \rangle &\rightarrow \langle nom^+.T^-, [nom^-, wh^-] \rangle \end{aligned}$$

Given a fixed MG G , one can compute all possible ways of unmerging and unmoving, and convert those to a finite collection of rewrite rules. Moreover, for each lexical item $A :: \alpha$ of G one adds a rewrite rule $\langle \alpha, [] \rangle \rightarrow A$. So the lexical item $John :: D^-.nom^-$ becomes the rewrite rule $\langle D^-.nom^-, [] \rangle \rightarrow John$. The result is a context-free grammar (CFG) that generates all the well-formed derivation trees of the MG G (and only those).

But generating syntactic derivations is not enough when it comes to discussing weak generative capacity, we need the actual string yields. This is where the CFG reaches its limits because it linearizes each mover in the positions where it enters the derivation, rather than its final landing site. In the concrete case of the derivation tree in (4), the CFG that gives rise to this derivation tree would produce the string *did John hit which car* instead of the intended *which car did John hit*. The CFG must be given some way to generate *which car* as intended but then linearize it in a different position from where it was introduced. This is exactly what one gets by taking the step from CFGs to multiple context-free grammars (MCFGs), but of course that is also what movement does in MGs: the overall shape of the mover is determined in its base position, but it is spelled out in its final landing site instead. Harkema (2001a) and Michaelis (2001) extend this intuitive sketch of MGs as CFGs with flexible linearization into a proof that MGs are weakly equivalent to MCFGs.

Several insights build on the weak equivalence proof of MGs and MCFGs (and note that it is truly the proof that provides these insights, the equivalence result itself is not nearly as informative). First of all, it shows that MGs are essentially context-free grammars with a more elaborate linearization procedure. This is similar to the original Aspects model (Chomsky 1965), where a context-free D-structure was rearranged by transformations — the transformational roots of Minimalism are still apparent in MGs. Second, Kobele (2010) proved that not all types of movement are the same in MGs. Head movement and remnant movement require the step from context-free to multiple context-free. Movement that results in a mover c-commanding all its traces in the output structure, on the other hand, only requires some smart coding tricks in order to be handled with the CFG sketched above. Hence an MG that only has this kind of *ECP-obeying phrasal movement* can generate only context-free languages.⁵ But this is insufficient because, as we already saw, Huybregts (1984) and Shieber (1985) showed that syntax is at least mildly context-sensitive. We

⁵ECP is short for the *Empty Category Principle*, which was a major restriction on movement in Government-and-Binding theory.

conclude that an empirically viable version of Minimalism cannot rely on ECP-obeying phrasal movement alone. Finally, Stabler (2013) points out that even though MGs and MCFGs are weakly equivalent, MGs are exponentially more succinct: an MG with n lexical items corresponds to an MCFG with up to 2^n rewrite rules. This again harkens back to the early days of Transformational Grammar, as Chomsky (1957) argued against CFGs not on the basis of their generative capacity but rather their inability to succinctly express generalizations. The same still holds over 60 years later: Minimalist syntax provides a much more compact description than (multiple) context-free rewrite rules.

The weak equivalence of MGs and MCFGs also allows us to roughly compare Minimalism to other frameworks, to assess how well the typology it predicts actually matches the class of natural languages, and how modifications to the formalism may affect its power. As discussed before and as is summarized in Fig. 19.1, MGs are more expressive than TAG (Joshi 1985; Joshi and Schabes 1997) and Combinatory Categorical Grammar (CCG) (Steedman and Baldridge 2003).⁶ At the same time, they are not completely unrestricted formalisms like HPSG or early versions of Transformational Grammar (Peters and Ritchie 1971, 1973b). Hence no additional assumptions (e.g. substantive universals or learnability requirements) are needed to rule out truly implausible languages such as the set of all strings whose length is prime. MGs, and by extension Minimalism, are roughly in the right ballpark.

It is curious that there seems to be no syntactic phenomenon that falls within the class of MCFLs but not the class of TALs. Total triplication in morphology would be an example of this, if it can be shown that the size of the triplicated material is indeed unbounded. But unless one wants to use MGs to model morphological reduplication and triplication, there seems to be no pressing need for the additional weak expressivity of standard MGs relative to TAG or CCG. However, in contrast to those formalisms, MGs allow for a straightforward extension that is in fact required for natural language: when movement may leave behind pronounced copies, MGs cross the threshold from MCFLs to parallel MCFLs (PMCFLs), and there are several language phenomena that apparently require this increased power. This includes Chinese number names (Radzinski 1991), case stacking in Old Georgian (Michaelis and Kracht 1997), and relativized predicates in Yoruba (Kobebe 2006). MGs are the only formalism on the market where the empirically motivated step from MCFLs to PMCFLs is almost trivial on a technical level and is also linguistically plausible through the copy-theory of movement.

It is not surprising, then, that the status of these PMCFL phenomena is contentious among

⁶Joshi et al. (1991) showed that CCG is weakly equivalent to TAG. However, this finding relies on machinery that is rarely used by the CCG community nowadays. Kuhlmann et al. (2015) show that a version of CCG that jettisons some of the mechanisms used by Joshi et al. (1991) while incorporating new machinery like typed slashes generates its own class of string languages that falls strictly between CFL and TAL, and that this may cause problems with the crossing dependencies in Dutch (but curiously, not for those found in Swiss German).

formal grammarians, prompting debates that resemble GPSG’s reaction to the crossing dependencies data. For instance, Bhatt and Joshi (2004) argue that Old Georgian case stacking could be reanalyzed as a simpler pattern that falls within the purview of TAGs. Along the same lines, the naturalness of the Chinese number names data has been doubted because the relevant patterns arise with very large numbers that are rarely encountered outside mathematical contexts. To the best of my knowledge, no counterargument has been presented to the Yoruba data in Kobele (2006). Nonetheless there is no shared consensus at this point that a formalism has to be able to generate PMCFLs in order to handle all aspects of syntax. This is an area where syntacticians could use their expertise to greatly strengthen the cases presented so far and even add new ones to the list. Because weak generative capacity arguments depend on such a small number of assumptions, they are a powerful addition to the Minimalist case for copy movement.

While copy movement has a significant impact on weak generative capacity, the same does not hold for the majority of proposals in the Minimalist literature. As MGs are such a bare-bones interpretation of Minimalism, they have been extended in numerous ways over the years: rightward movement (Graf 2012a), sideward movement (Stabler 2006), across-the-board movement (Kobele 2008; Torr and Stabler 2016), lowering movement (Graf 2012a), clustering movement (Gärtner and Michaelis 2010), adjunction (Frey and Gärtner 2002; Fowlie 2013; Graf 2014b; Hunter 2015), TAG-style tree adjunction (Graf 2012b), Late Merge, (Graf 2014a), phases (Stabler 2011a), and transderivational constraints (Graf 2013), among others. None of them increase the weak generative capacity of the formalism — at the level of strings, they can be emulated by the basic MG version in Sec. 2.

This emphatically does not imply that all these proposals are linguistically equivalent, but rather that any differences between them cannot be related to the well-formedness of surface strings. There may still be differences in the structural descriptions, the mapping to semantics or prosody, or the succinctness of the grammar. Weak equivalence does not mean that there is no point in further comparing these different proposals, but that any empirical data that is taken to differentiate between them only does so when embedded in a network of additional syntactic assumptions. If these assumptions change, so might the relation between the proposals.

It is also worth keeping in mind that these results have only been established for MGs with the SMC. As far as I can tell, they should also hold for the MG variants with relaxed SMCs that I will discuss in the next section. But if those variants should still prove too restrictive for natural language, if the SMC and anything like it needs to be excised from MGs, then there is no telling how the additions and modifications studied so far affect weak generative capacity. MGs without any SMC-like constraints are known to have greater weak generative capacity than standard MG (Salvati 2011), but the real problem is that such MGs behave in unexpected ways. For example, adding the ban against left branch extration (also called the *Specifier Island Constraint*) to MGs with the

SMC reduces their expressivity to so-called monadic branching MCFLs (Michaelis 2009; Kobele and Michaelis 2011). But adding the very same constraint to MGs without the SMC turns them into a completely unrestricted formalism that can generate any recursively enumerable stringset (Kobele and Michaelis 2005; Gärtner and Michaelis 2007). While overgeneration is not as much of an issue as undergeneration, and while some of the slack could be picked up by substantive universals as well as processing and learnability considerations, generating all recursively enumerable languages reduces MGs to the status of a pure description language that puts no meaningful restrictions at all on what is a possible natural language. This is yet another case where weak generative capacity, whatever its linguistic shortcomings may be, helps us pinpoint the parts of the formalism that keep it closely aligned with natural language syntax and should be preserved if at all possible: the SMC (or some comparable collection of locality constraints) is an indispensable cap on the power of any system with Merge and Move.

3.3 Cliffs notes

While weak generative capacity is largely considered irrelevant by syntacticians, it does offer insights of linguistic interest. The mild context-sensitivity of natural language gives us a lower expressivity threshold that any descriptively adequate theory of syntax must pass. Minimalism, in its incarnation as MGs, succeeds at that, provided that some form of remnant movement or head movement is available — ECP-obeying phrasal movement on its own is not enough to push MGs beyond the context-free boundary. Once remnant movement and/or head movement is available, MGs are weakly equivalent to MCFGs, although they are exponentially more succinct than MCFGs (a modern incarnation of Chomsky’s original argument against phrase structure grammars). The weak equivalence to MCFGs suggests a close connection to CFGs, and indeed every MG can be regarded as a CFG with a more complex linearization algorithm. The CFG generates the well-formed MG derivation trees, which are then linearized in a fashion that takes movement into account, very much like the Aspects model uses a phrase structure grammar to generate D-structures that are then modified by transformations. Interestingly, the Minimalist proposal of copy-movement opens up a path beyond MCFGs so that purported PMCFL phenomena like Old Georgian case stacking and relativized predicates in Yoruba can be analyzed as an instance of copy-movement with multiple pronounced copies. Competing formalisms either cannot reach this level of expressivity without major modifications (TAG and CCG), or they are so powerful that the formalism on its own imposes no meaningful restrictions on natural languages (HPSG). MGs thus hit a sweet spot with respect to weak generative capacity, provided that Merge and Move are adequately restricted via locality constraints, which is guaranteed by the SMC in standard MGs.

4 Strong generative capacity and the malleability of MGs

From the perspective of weak generative capacity, MGs get a lot right, but their behavior depends on the SMC, which seems at odds with Minimalist practice. MGs may also appear too far removed from modern Minimalist syntax, the various extensions mentioned above notwithstanding. Once the study of MGs shifted from weak generative capacity to **strong generative capacity**, these issues could suddenly be addressed in an elegant manner because MGs turned out to be much more malleable than their original definition suggests.

In this section, we will see that MGs in particular and Minimalist syntax in general can be reinterpreted as a constraint-based formalism. As long as these constraints are stated in a specific extension of first-order logic (Sec. 4.1), they do not increase expressivity relative to standard MGs. That is because there are ways to automatically compile the constraints into the lexicon of a standard MG. This general setup makes it possible to effortlessly adapt various proposals from the Minimalist literature, secure in the knowledge that the computational properties of the formalism remain the same because the constraints can be lexicalized out of existence (Sec. 4.2). The constraint-based perspective of MGs opens up a diverse yet tightly controlled space of options in which MGs can be revamped from the ground-up, changing even core assumptions like the SMC (Sec.4.3) in order to make MGs as faithful to Minimalist syntax as one desires.

4.1 Logic for constraints and mappings

Strong generative capacity measures the complexity of the structural descriptions a formalism can provide. But there are many types of structural descriptions: phrase structure trees, dependency trees, derivations, syntax-LF mappings, syntax-PF mappings, and possibly more. As we saw in Sec. 2.3, though, derivation trees provide a baseline for MGs from which these other types of structures are produced. This dovetails nicely with the fairly recent realization in formal language theory that grammar formalisms that generate (P)MCFLs or TALs can be decomposed into two components: I) a **monadic second-order (MSO)** definable tree language, and II) an MSO-definable mapping from those trees to the desired structural description (Morawietz 2003; Mönnich 2006). In the case of MGs, these two components correspond to the set of well-formed derivation trees and the procedure that translates derivation trees into the desired output structures. This sounds very abstract, but it only takes a minimal amount of familiarity with MSO in order to make this idea more concrete.

MSO is an extension of first-order logic with quantification over sets. Even though linguists tend to think of logic as a semantic tool, it is also very well-suited as a description language for structures and mappings. In the following, let us consider some examples of MSO-formulas that, taken together, will add a basic version of Agree to MGs. The general strategy is to take the set of

derivation trees generated by this MG, and then use MSO to infer additional information that can be layered on top of those trees.

The Agree operation usually involves features that are not part of lexical items in MGs, e.g. tense features and ϕ -features. As our first step, then, we have to add those features. Following Minimalist thinking, only valued features should be fully specified in the lexicon. For example, the entry $\text{sheep} :: N^-$ will now be split in two, one being $\text{sheep} :: N^-[3\text{sg}]$, the other being $\text{sheep} :: N^-[3\text{pl}]$. Morphological specifications like $[3\text{sg}]$ are purely descriptive placeholders here, and any given feature theory of person and number (see e.g. Harley and Ritter 2002; Nevins 2007; Adger 2010; Svenonius 2019) will do for our purposes. The reason the choice of feature system does not matter is that we will immediately layer our own, MSO-based feature system on top of the lexical system. We do this by defining predicates like $3_v(x)$ that hold of x iff x is one of the finitely many lexical items that are third person.

(7) **Defining a logical predicate for lexically valued third person**

$$3_v(x) \Leftrightarrow \text{sheep} :: N^-[3\text{sg}](x) \vee \text{sheep} :: N^-[3\text{pl}](x) \vee \text{who} :: D^-. \text{wh}^-[3\text{sg}](x) \vee \text{destruction} :: P^+. N^-[3\text{sg}](x) \vee \dots$$

“A node x in a derivation tree has lexically valued 3rd person iff x is $\text{sheep} :: N^-[3\text{sg}]$ or $\text{sheep} :: N^-[3\text{pl}]$ or $\text{who} :: D^-. \text{wh}^-[3\text{sg}]$ or $\text{destruction} :: P^+. N^-[3\text{sg}]$ or ...”

This logical formula defines a new predicate 3_v that holds of all lexical items, and only those, that enter the derivation with a valued third person feature. We can use the same basic trick of listing all relevant lexical items (of which there can be only finitely many) to define a predicate that picks out nodes with unvalued person features.

(8) **A predicate for unvalued person features**

$$\text{pers}_u(x) \Leftrightarrow \text{sleep} :: D^+. V^-(x) \vee \text{sleeps} :: D^+. V^-(x) \vee \text{like} :: D^+. D^+. V^-(x) \vee \dots$$

“A node x in a derivation tree has an unvalued person feature iff x is $\text{sleep} :: D^+. V^-$ or $\text{sleeps} :: D^+. V^-$ or $\text{like} :: D^+. D^+. V^-$ or ...”

Note that pers_u is essentially a new feature, but one that did not come from the lexicon. Rather than lexicalizing the feature, we used a logical formula to automatically infer the feature. This technique is very powerful, and is exactly why the deviations of the MG feature calculus from mainstream Minimalism are not as serious as they might seem: one can always layer a different feature system on top of MGs, as we are doing now with unvalued features. Using the same kind of listing technique, we can define valued features for person (1_v , 2_v , 3_v), number (sg , pl), tense (*past*, *present*, *future*, ...), and much more, and we can also define unvalued counterparts such as pers_u , num_u and tense_u .

With all those logical predicates for valued and unvalued features, we may get started on the Agree operation. For the sake of simplicity, we will ignore how movement interacts with Agree —

the MSO approach makes it remarkably easy to incorporate movement, but it would still require too many formulas to fit here. In this movement-free version of Agree, a lexical item with an unvalued feature will probe its c-command domain for lexical items with a valued feature of the appropriate type, picking the value of the closest feature.

We will express this idea through multiple MSO formulas. Let us assume that we already have a predicate $c\text{-com}(x, y)$ to express that x c-commands y (see e.g. Rogers 1998 for how this predicate can be defined in MSO from the maximally local mother-of relation). Next we make use of MSO's ability to quantify over sets of nodes, which is what sets it apart from standard first-order logic. For number agreement, for example, we will require the existence of two sets **sg** and **pl**, each one of which contains all nodes with this number feature, irrespective of whether the node got it through its lexical specification or via Agree. More precisely, each one of **sg** and **pl** is defined inductively to contain all nodes with sg_v and pl_v , respectively, and to contain every node that has an unvalued number feature and is the closest c-commander of a node in the respective set.

(9) **Number agreement via set quantification**

$$\exists \mathbf{sg} \exists \mathbf{pl} \forall x \left[\left(\mathbf{sg}(x) \leftrightarrow sg_v(x) \vee \exists y [num_u(x) \wedge \mathbf{sg}(y) \wedge c\text{-com}(x, y) \wedge \neg \exists z [\mathbf{pl}(z) \wedge c\text{-com}(x, z) \wedge c\text{-com}(z, y)]] \right) \wedge \left(\mathbf{pl}(x) \leftrightarrow pl_v(x) \vee \exists y [num_u(x) \wedge \mathbf{pl}(y) \wedge c\text{-com}(x, y) \wedge \neg \exists z [\mathbf{sg}(z) \wedge c\text{-com}(x, z) \wedge c\text{-com}(z, y)]] \right) \wedge \neg (\mathbf{sg}(x) \wedge \mathbf{pl}(x)) \right]$$

“There are two sets of nodes, called **sg** and **pl**, such that I) node x is contained in **sg** iff x has a lexically specified singular person feature, or x needs a number feature and c-commands a node y that belongs to **sg** and there is no intervener z belonging to **pl**, and II) node x is contained in **pl** iff analogous conditions are satisfied, and III) no node may be both in **sg** and in **pl**.”

We have to define analogous sets for every type of feature that we want to be passed around via Agree, i.e. number, gender, and so on. They all follow the same basic template above, though, and thus Agree still presents a unified concept that could be expressed by a single meta-formula that is instantiated in a specific way for each feature type. Note that (9) isn't a definition but a constraint: it requires every derivation tree to be such that these sets can be correctly instantiated. In other words, Agree is recast as a constraint on how valued and unvalued features may be distributed over the derivation tree.

At this point we have an MSO-based implementation of how Agree passes feature values from one lexical item to another, which in turn may feed future applications of Agree. Note that the linguistic concept of features is rendered in three different ways: as part of a lexical item's specification, as a logical predicate like 3_v , and as a set of nodes like **3**. While we could have had a more unified implementation, this solution emphasizes how fluid the notion of features is on a formal

level, and that one can easily shift information from the lexicon into logical formulas. But our task is still not quite done: it is not enough to pass around features, we also have to capture how feature passing via Agree affects the surface morphology.

There are two routes here, one via constraints, the other via MSO-based spell-out rules. Suppose the MG lexicon contains fully inflected forms, for instance both *sleep* :: $D^+.D^+.V^-$ and *sleeps* :: $D^+.D^+.V^-$. Then morphological realization may take the form of a filter: we ban derivations where the inflected form is incompatible with the features that were inherited via Agree. This does not work if the lexicon only lists uninflected forms, though. Fortunately, this case is covered by MSO’s ability to also define mappings from one structure to another, in particular from a derivation tree to a phrase structure tree.

In the case at hand, we would add output predicates (indicated by a subscripted $_o$) that tell us how a given lexical item ought to be pronounced in the output structure. For example, if node x is *sleep* :: $D^+.D^+.V^-$, then it would be spelled out as *sleeps* if x is a member of **3** and **sg** and **present**, and as *sleep* if it is not in **3** or not in **sg**.⁷

(10) **Spell-out formulas for uninflected base forms**

- a. $sleeps_o(x) \Leftrightarrow sleep :: D^+.D^+.V^- \wedge \mathbf{3}(x) \wedge \mathbf{sg}(x) \wedge \mathbf{present}(x)$
- b. $sleep_o(x) \Leftrightarrow sleep :: D^+.D^+.V^- \wedge \neg(\mathbf{3}(x) \wedge \mathbf{sg}(x)) \wedge \mathbf{present}(x)$

Output predicates can have much more complex definitions. For example, the definition could include information about the syntactic context to determine the surface forms of roots as in Distributed Morphology, we could flatten parts of structure into a single lexical node as in Nanosyntax, or we could use string adjacency to capture the *a/an* alternation in English. We can also define new relations like “mother-of in the output”, and this is how MSO can move phrases from their base position to their final landing site. Quite generally, MSO is a very powerful toolbox that makes it very easy to express linguistic ideas over MG derivation trees.

Our implementation of Agree is admittedly lacking in several linguistic respects: movement is not taken into account, features probe independently of each other rather than as a single bundle, there are no locality domains or defective intervention, and we do not handle more complicated edge cases like omnivorous number (Nevins 2011). But the implementation is rich enough to illustrate the core techniques behind MSO, and how this logic makes it possible to layer additional mechanisms on top of standard MGs.⁸ In fact, Rogers (1998) even gives a full MSO-implementation of GB as presented in Haegeman (1994), and if one desired so one could use MSO

⁷Since these definitions use **3**, **sg**, and **present**, they must appear in the scope of the existential quantifiers that instantiate those sets. This is omitted here for the sake of brevity and readability.

⁸Ermolaeva (2018) and Ermolaeva and Kobele (2022) present an alternative implementation in which the checking of Merge and Move features creates information channels through which morphological information can be passed between lexical items. This proposal exemplifies how MGs and their explicit feature calculus can give rise to new styles of analysis.

to enrich MGs with all that GB machinery, from case filters to theta grids. The overall strategy is always the same: syntactic constraints are easily converted to MSO-constraints, and syntactic operations are easily converted to a combination of MSO-constraints and MSO-mappings (expressed via output predicates).

4.2 Minimalism as a constraint-based theory

With this MSO foundation, we can now make sense of the claim in Sec. 3.2 that most MG extensions do not affect weak generative capacity. It is known that an MSO-definable tree language with an MSO-definable mapping yields at most MCFLs (or PMCFLs if the MSO-mapping is combined with a copying mechanism). MGs already generate (P)MCFLs, putting them at the expressive edge of what is possible in this MSO-based setup. Consequently, any extension that can be expressed as an MSO-constraint, an MSO-mapping, or a combination of the two, cannot push MGs beyond (P)MCFLs.

But virtually all Minimalist proposals in the literature are MSO-definable in this sense. Sometimes the translation to MSO is obvious. For example, it is fairly easy to define an MSO-constraint that bans movement of a lexical item with f^- if there is another lexical item closer to the target site that could have carried f^- instead — a kind of Relativized Minimality.⁹ Sometimes the translation is very involved. The transderivational constraints from early Minimalism, for example, are optimality conditions that compare multiple trees, whereas MSO can only consider individual trees. But Graf (2013) shows that for any given MG G , transderivational constraints like Merge-over-Move can be translated to a local constraint on individual derivation trees generated by G , and this constraint is definable in MSO. Finally, there are a few proposals that seem to go beyond the power of MSO, most notably those that involve semantics, a point I will return to in a moment. These few outliers notwithstanding, the general picture that has emerged from over two decades of MG research is that virtually all Minimalist proposals in the literature are MSO-definable — for the same reason that Rogers (1998) could define even the most arcane details of GB in terms of MSO — and this is why these proposals do not increase the weak generative capacity of MGs.

Of course one should not infer from this that any combination of an MSO-tree language with an MSO-mapping is a possible MG. To wit, TAGs are also the combination of two such MSO-components, yet they differ from MGs in many important ways. Not only is TAGs' weak generative

⁹First, we once again use the listing technique to define a predicate F that picks out all lexical items that do not carry the licensee feature f^- but could have. Then, we change the definition of mover so that it also contains lexical items with predicate F . Hence, if we have a lexical item x with f^+ and a possible mover y with f^- yet some lexical item z between the two satisfies predicate F , y is not allowed to move to x because it is not the closest possible mover, but at the same time z cannot check f^+ on the target site because it lacks f^- . This causes an irresolvable feature checking conflict that renders the derivation ill-formed.

capacity less than that of MGs, they also generate a very different class of phrase structure trees,¹⁰ and their parsing algorithms (Kallmeyer 2010) differ noticeably from those for MGs (Harkema 2001b; Stabler 2013; Stanojević and Stabler 2018). MSO is a very large class, so the fact that MGs occupy a section of that class does not mean that everything in the class is an MG. Just like weak generative capacity provides key insights but does not tell the full story, the MSO-nature of MGs explains a lot of their properties but is not the formal essence of MGs.

That said, many MG extensions are not only MSO-definable, they can be fully reduced to standard MGs. This entails that these extensions do not affect strong generative capacity either; in fact, they are just a convenient shorthand for defining standard MGs. That is because MSO-constraints have a principled connection to the MG feature calculus: every MSO-constraint can be compiled into the feature calculus (Kobele 2011; Graf 2011), and the whole MG feature calculus can be abstracted into MSO (Graf 2017). As far as weak and strong generative capacity are concerned, there is no difference between a purely feature-driven MG as in Sec. 2 and a purely constraint-based one where operations apply freely in a generate-and-filter manner (and this is also why the differences between MG features and Minimalist features are largely innocuous).

We have already seen in Sec. 4.1 how MSO can layer its own features over the derivation tree via logical predicates or as set quantification, so let us briefly explore the other direction of converting MSO-constraints into features. Consider once more the lexicon in (1), repeated here as (11).

(11) **A small MG fragment for English**

car :: N^-	the :: $N^+.D^-$	hit :: $D^+.D^+.V^-$
John :: D^-	the :: $N^+.D^-.nom^-$	ε :: $V^+.nom^+.T^-$
John :: $D^-.nom^-$	which :: $N^+.D^-.wh^-$	ε :: $T^+.C^-$
	which :: $N^+.D^-.nom^-.wh^-$	did :: $T^+.wh^+.C^-$

Now suppose that we add the (linguistically preposterous) MSO constraint that objects must consist of an even number of nodes and subjects of an odd number. This involves defining two sets **O** and **E** such that a node is in **O** iff its subtree has an odd number of nodes, and **E** otherwise. We then require that the root of a object must be a member of **E**, and the root of a subject must be a member of **O**.¹¹ We can enforce this constraint directly in the lexicon by splitting the category feature **D** into

¹⁰Standard MGs generate multiple regular tree languages (Mönnich 2006, 2007) whereas TAGs generate monadic linear context-free tree languages (Kepser and Rogers 2011). At this point it is unclear which one of the two classes is a better fit for natural languages. Graf (2012b) shows that TAGs can be translated to MGs where standard phrasal movement is replaced a specific kind of lowering movement. Alternatively, one can also use a specific kind of sideward movement. If this kind of movement were to be shown to be inadequate for syntax, that would constitute structural evidence against TAG.

¹¹The full constraint is $\exists \mathbf{O} \exists \mathbf{E} \forall x, y, z \left[\left(\mathbf{O}(x) \leftrightarrow \neg \mathbf{E}(x) \right) \wedge \left(leaf(x) \rightarrow \mathbf{O}(x) \right) \wedge \left(Move(x) \wedge mother-of(x, y) \rightarrow \left(\mathbf{O}(x) \leftrightarrow \neg \mathbf{O}(y) \right) \right) \wedge \left(Merge(x) \wedge mother-of(x, y) \wedge mother-of(x, z) \wedge \neg y \approx z \rightarrow \left(((\mathbf{O}(y) \wedge \mathbf{O}(z)) \vee (\mathbf{E}(y) \wedge \mathbf{E}(z))) \leftrightarrow \right. \right. \right.$

two subtypes D_e and D_o . Even though this split is linguistically dubious, it is comparable to the split between mass nouns and count nouns on a mechanical level. With D_e and D_o , we can now replace the verb *hit* :: $D^+.D^+.V^-$ with *hit* :: $D_e^+.D_o^+.V^-$. As a result, objects must have an even number of nodes and subjects an odd number, exactly the condition expressed by our MSO-constraint. The complete refined lexicon is shown below.

(12) **The same fragment but with D split into two subcategories**

<i>car</i> :: N^-	<i>the</i> :: $N^+.D_e^-$	<i>hit</i> :: $D_e^+.D_o^+.V^-$
<i>John</i> :: D_o^-	<i>the</i> :: $N^+.D_e^-.nom^-$	ε :: $V^+.nom^+.T^-$
<i>John</i> :: $D_o^-.nom^-$	<i>which</i> :: $N^+.D_e^-.wh^-$	ε :: $T^+.C^-$
	<i>which</i> :: $N^+.D_e^-.nom^-.wh^-$	<i>did</i> :: $T^+.wh^+.C^-$

The translation is rarely as intuitive as in the example above, but it can always be carried out in a purely algorithmic manner that requires no understanding of what the MSO-constraints actually do in the grammar.

We conclude, then, that features can be shifted from the lexicon into MSO-constraints, and MSO-constraints can be compiled into the lexicon via features. This is the mathematical state of affairs, but does it hold up from a linguistic perspective? There are definitely concerns about the linguistic plausibility of refining category features in this manner, but Graf (2017) argues that linguistics currently lacks a sufficiently rigorous definition of syntactic categories to prevent such feature coding. Syntactic categories are posited based on a variety of factors, in particular morphological inflection, semantics, and syntactic distribution. Each factor on its own is insufficient to rule out a split like D_e and D_o — they describe tendencies, not hard restrictions. But there is no explicit theory about how these factors are weighted relative each other, and hence their combination cannot rule out such feature coding, either. It is also important to keep in mind that the outcome of feature coding is rarely as neat and obvious as in the example above. As the complexity and number of MSO-constraints increases, it becomes increasingly difficult to decipher the resulting lexicons. The informal notions of syntactic category cannot adequately be applied to such complex lexicons. As we will see in Sec. 5.1, there are now mathematical attempts to explain what separates an intuitively plausible lexicon like the one in (11) from the unnatural (12). But this still does not change the fact that there is a mathematical equivalence between features and MSO-constraints, even though the equivalence may not be fully realizable unless one allows for very unnatural lexicons. With respect to syntactic proposals that extend the core of Minimalism in a manner that is definable in MSO, this equivalence then tells us at least the following: whatever these proposals

$O(x)) \wedge (subject(x) \rightarrow O(x)) \wedge (object(x) \rightarrow E(x))$], where *mother-of*(x, y) holds iff x is the mother of y , *leaf*(x) is a shorthand for $\neg \exists y[mother-of(x, y)]$, and $x \approx y$ is true iff x and y are the same node. The predicates *object* and *subject* are also defined in terms of *mother-of*, but I omit this here.

add in terms of expressivity could also be gained by giving up whatever constraints one would like to put on syntactic categories.

Note furthermore that this equivalence only holds for MSO-constraints — changes to the MSO mapping might still affect what kind of output structures can be produced. But if one takes seriously the notion that the core of syntax is about the application of Merge and Move, then the central data structure for syntax should be derivations, not the output of those derivations. And once our focus is squarely on derivations, all Minimalist proposals correspond to MSO-constraints on MG derivation trees. We already encountered this in our treatment of Agree in Sec. 4.1: even though Agree is usually thought of as an operation, we were able to implement it as a constraint on the distribution of valued and unvalued features in derivation trees, and only the post-syntactic spell-out rules were defined as a mapping instead of a constraint. Similarly, sideward movement just relaxes the constraints on how licensor and licensee features may be distributed over the derivation tree, offering the option of feature checking via movement in the absence of c-command (Graf 2012a). MSO effectively reinterprets all of Minimalism as a constraint-based theory of derivation trees, and in doing so it allows us to quickly and safely extend MGs in numerous ways because MSO-constraints do not add anything that we cannot already enforce through the feature calculus of standard MGs.

The number of papers studying MG extensions has decreased significantly in recent years exactly because the MSO perspective makes it so easy to extend MGs while preserving both weak and strong generative capacity (when the latter is construed in terms of derivations). At this point, MGs are so malleable that very few Minimalist proposals cannot be made to fit. The most problematic cases are constraints that make reference to perfect identity of meaning, e.g. Rule I (Reinhart 1983; Heim 1998) and some accounts of ellipsis (see Kobele 2015 for a discussion of ellipsis informed by MGs). The challenge with these constraints is that identity of meaning is an undecidable property in any semantic theory that is at least as expressive as first-order logic. But even in those cases the notion of identity of meaning might just be a convenient shorthand for an intuitively more complicated but computationally less complex concept, a concept that might be expressible in MSO. Overall, then, MGs turn out to be a much more flexible toolbox than the sparse definition might suggest: All MSO-definable constraints are reducible to standard MGs that only use Merge and Move, and in a world where syntax is about derivations, almost all Minimalist proposals are MSO-definable constraints.

4.3 Relaxing the SMC

MSO also provides the means for relaxing the SMC, which is the biggest hurdle towards implementing Minimalist analyses in a maximally faithful manner. Consider once more the example

of *who likes what* from the end of Sec. 2. The problem was that a faithful analysis should posit two wh-features, one on *who*, the other on *what*, and both features should be active at the same time, which violates the SMC. With MSO, one can weaken the SMC to get around this problem. For instance, one could say that licensee features are checked if at all possible but may remain unchecked if that violates other locality constraints, including the SMC. Or one could say that *who likes what* contains only a single wh-feature, but there are locality constraints that forbid this wh-feature from occurring on *what* instead of *who*. In both cases, the resulting grammar could still be automatically converted to a standard MG, making these two strategies just different ways of defining standard MGs in a manner that is closer to linguistic conventions.

If one is willing to trade in reducibility to standard MGs for even more linguistic faithfulness, the SMC can be relaxed even further (while avoiding the overgeneration issues of MGs without the SMC that were mentioned at the end of Sec. 3.2). In the MSO-framework, the primary role of the SMC is to guarantee two properties of MGs: that the set of well-formed derivations is MSO-definable, and that there be no ambiguity as to what moves where. In MGs without the SMC or some comparable replacement, neither property holds. But that is not to say that the SMC is indispensable. The SMC can be removed without losing those two properties if one also adds a collection of MSO-definable locality constraints on Move. These MSO-constraints do not need to be as rigid as the SMC and can instead be closely modeled after linguistic constraints such as Relativized Minimality. Once such an SMC-less system is in place, we can also allow licenser and/or licensee features to be checked more than once, so that a single mover can undergo an unbounded number of feature-driven movement steps, e.g. for raising, or a single landing site can be targeted by an unbounded number of movers, e.g. for multiple wh-movement (Stabler 2011a; Laszakovits 2018; Graf and Kostyszyn 2021). Because the resulting system is still MSO-definable, these extended MGs have the same weak generative capacity as standard MGs, but the removal of the SMC means that derivations may be more expressive.

This leaves us with two key insights from the MSO-perspective of MGs. First, standard MGs are a lot closer to Minimalism than their barebones definition would suggest. Almost all Minimalist proposals correspond to MSO-constraints on MG derivations, which are the central data structure of MGs. Hence these proposals can already be expressed by the feature calculus of standard MGs, although the result is much less elegant than the MSO-formulas. Only a handful of analytic options require more drastic changes because they are at odds with the SMC. But MSO can safely replace the SMC with a more flexible system without losing many of the core properties of MGs. The constraint-based view of MGs and Minimalism thus has greatly expanded our understanding of what the formalism is capable of.

4.4 Cliffs notes

There is a lot of work on MGs as generators of trees rather than strings. In particular, derivation trees have emerged as the central MG data structure from which various output structures can be obtained in an automatic manner. This coincides with a recent trend in formal language theory to analyze mildly context-sensitive grammar formalisms like TAGs and MGs as consisting of two components: a tree language definable in MSO, and a mapping from those tree to the desired output trees. MSO is a powerful extension of first-order logic with the ability to also quantify over sets of nodes. Standard MGs have an MSO-definable derivation tree languages and an MSO-definable mapping from derivation trees to phrase structure trees. The power of MSO makes it easy to layer various linguistic proposals on top of standard MGs, enriching them with Agree, adjunction, locality constraints, and various types of movement. One can even add completely new features, as we saw in our implementation of Agree in Sec. 4.1. Since these additions stay within the realm of MSO, they cannot increase the weak generative capacity of MGs, which is already at the very peak of what is possible with MSO-definable formalisms. What more, there is an automatic procedure for compiling MSO-constraints directly into the MG lexicon, and hence any linguistic proposal that can be captured purely via MSO-constraints could already be expressed in standard MGs — adding it to the formalism affects neither weak nor strong generative capacity. As a result, MGs are much more flexible than it seems at first glance.

For linguists, this has several implications: First, MG results should not be dismissed as inapplicable to Minimalism due to MGs being a seemingly impoverished version of Minimalism. Quite to the contrary, MGs identify a computational core of Minimalism that remains stable even under significant modification. Second, syntacticians need not worry too much about overgeneration. It is very unlikely that a new constraint would push us beyond MG’s (P)MCFL horizon. But at the same time, syntacticians should worry a lot about overgeneration because the simple feature checking mechanism driving Merge is already enough to give us the full power of MSO, which is much more than we need for syntax. The next section surveys recent attempts to curtail this overgeneration.

5 Subregular complexity: syntax $\approx$ phonology?

The MSO perspective has made MGs a tremendously malleable toolbox that easily accommodates most Minimalist machinery. This analytical flexibility is desirable for computational projects that wish to incorporate Minimalist proposals and use MGs as computational wrapper around Minimalism to get the level of mathematical rigor they require for their models (some of those will be discussed in the next section). But it is also a worrisome state of affairs that has prompted a more

fine-grained approach to MGs in recent years.

The extreme flexibility of MGs reveals them, and by extension Minimalism, to be a much looser characterization of natural language than one would assume. Remember, the duality of features and MSO-constraints means that every MSO-definable constraint can be enforced by the basic MGs we saw in Sec. 2. This includes many constraints that are natural from a Minimalist perspective, and many more that are not. For example, MSO can distinguish between odd and even by instantiating two distinct sets **O** and **E** (see also fn. 11). Yet there seem to be no syntactic constraints that require distinguishing an even number of nodes from an odd one. Graf (2017) discusses several examples of MSO-based overgeneration, each one more unnatural than the previous one. Some of the MSO-constraints also punch big loopholes into syntactic constraints, e.g. island constraints, depriving them of all empirical content. As a short demonstration of how bad things are, the reader is invited to write down an arbitrary MSO formula, with quantifiers and logical connectives combined in a random fashion. As long as the result obeys the syntax of MSO and contains no free variables, it is a constraint that can be enforced by standard MGs. But odds are this random MSO-formula has nothing in common with any constraint known to syntacticians — MSO is a giant class, and natural constraints are but tiny isles in this vast ocean of the unnatural.

Due to the expressive equivalence of features and MSO-constraints, fixing this problem requires a two-pronged approach that limits both the power of features and the power of constraints. Limiting only features accomplishes nothing because we still have the full arsenal of MSO-definable constraints at our fingertips. Limiting only constraints accomplishes nothing because we can hide the constraints in the feature system. We need restrictions on features as well as constraints.

5.1 Limiting features

Let us first consider ways to limit the power of features. The problem with MG features is that they have no computational cost attached to them. Recall from Sec. 2 that the lexicon already contains fully feature-annotated items, with no attention being paid to where those feature annotations come from. Consider once more the refined lexicon in (12), repeated here as (13).

(13) MG lexicon with **D** split into two unnatural subcategories

car :: N^-	the :: $N^+.D_e^-$	hit :: $D_e^+.D_o^+.V^-$
John :: D_o^-	the :: $N^+.D_e^-.nom^-$	ε :: $V^+.nom^+.T^-$
John :: $D_o^-.nom^-$	which :: $N^+.D_e^-.wh^-$	ε :: $T^+.C^-$
	which :: $N^+.D_e^-.nom^-.wh^-$	did :: $T^+.wh^+.C^-$

As far as MGs are concerned, (13) is just as valid a lexicon as the original one with a single **D**-feature, even though the former enforces the unnatural constraint that objects must have an even

number of nodes and subjects an odd number. As we discussed in Sec. 4.2, this is difficult to do by considering each syntactic category in isolation, which is what the usual criteria of morphological inflection, semantics, and syntactic distribution urge us to do. An alternative route is to focus not on individual features, but on the nature of the feature system as a whole. Ideally, this should give us a mathematical, substance-neutral criterion for distinguishing the natural lexicon with a single D-feature from its unnaturally refined counterpart in (13) with both D_o and D_e .

In this spirit, Graf (2020) proposes that Merge features, which are the primary source of the feature-constraint equivalence, have to be inferable in a local manner. Suppose, for instance, that an MG derivation tree like the one in (3) lacks all feature annotations. Would it be possible to at least infer the correct category and selector features for each lexical item? Graf (2020) argues that this is indeed the case, and that very little needs to be considered to determine a lexical item's category and selector features: the lexical item itself, the heads of the arguments it takes, and the head that takes it as an argument.

The underlying intuition is conveyed by the example of English *water*. In isolation, this could be a noun or a verb. But the verb takes a DP argument and is selected by a functional head, whereas the noun lacks an argument and is selected by a (possibly empty) D-head. This information is readily available from the local context of *water* in the derivation. For example, we may see that *water* takes an argument with head *the*, which must be a determiner and thus *water* is taking a DP-argument, clearly identifying it as a verb. Now contrast that to distinguishing D_o from D_e . For the small grammar in (12), that is indeed doable by looking only at the D-head itself and whether it takes an argument. But suppose that we add the lexical items below to (12) in order to allow for recursive possessor constructions.

(14) **Lexical items for possessor-recursion**

$$'s :: N^+.D_e^+.D_e^- \quad 's :: N^+.D_o^+.D_o^-$$

Our grammar now allows for DPs with *John*, which always have an odd number of lexical items (e.g. *John*, *John's car*, or *John's car's car*), and for DPs with *the* that always have an even number of lexical items (e.g. *the car*, *the car's car*, and *the car's car's car*). The only way to tell whether the possessive marker *'s* has category feature D_o or D_e is to fully inspect its complement and its specifier, no matter how large they are. In other words, if the unnatural even-odd counting constraint were to be lexicalized, correct assignment of the resulting category features would require the ability to inspect whole subtrees of arbitrary size, which conflicts with the generalization in Graf (2020) that category features in natural languages can be inferred by inspecting just the local head-argument relations.

A lot of empirical work remains to be done before we can safely conclude that Graf's locality domain is empirically adequate across all languages. In addition, the precise size of the domain

will vary depending on one’s theoretical assumptions, e.g. whether syntax contains fully inflected forms as in early Minimalism or DM-style roots. But as was already hinted at in the previous example, the central insight is not so much how large the locality domain is, but that there seems to be some locality domain of fixed size. This is probably true, in which case the local inference requirement greatly limits what kind of constraints can be expressed via the MG feature calculus, limiting one source of unnaturalness in MGs.

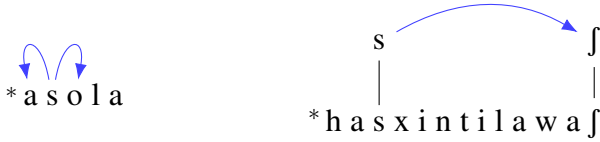
5.2 Limiting constraints

This leaves us with our second task: limiting the power of constraints. Here it has proven useful to borrow ideas from the field of **subregular phonology** (see Heinz 2009, 2018, Chandlee 2014, Jardine 2016, and references therein). Subregular phonology was born out of the desire for a tighter computational characterization of phonological phenomena. While phonology is commonly modeled in terms of regular languages and finite-state machinery (see Fig. 19.1), this poses exactly the same overgeneration issues as the MSO-view of syntax. For instance, one could define highly unnatural variants of vowel harmony where harmony applies only if the word contains an even number of vowels, or where neutral vowels block harmony if the word both starts and ends with a neutral vowel. The desire to curtail overgeneration prompted the development of weaker subclasses of the regular languages (hence the term **subregular**), and one of the most surprising findings in recent years is that the classes that play a major role in phonology are also tremendously useful for syntax.

It quickly became clear that most phonotactic phenomena are either **strictly local** or **tier-based strictly local** (Heinz et al. 2011; De Santo and Graf 2019), and that these classes are also connected to Merge and Move. A phonotactic constraint is strictly local iff it involves string contexts of a finitely bounded size.¹² Intervocalic voicing, for example, is strictly local because it only involves string contexts of three adjacent segments — to be more precise, contexts where one segment is flanked by two vowels. A phonotactic constraint is tier-based strictly local iff it is strictly local over a tier (inspired by but not identical to tiers in Goldsmith 1976). Take Samala sibilant harmony, where a word may not contain any two sibilants that disagree in anteriority, no matter how far apart those sibilants are (Applegate 1972). This is not strictly local, but it is strictly local over a sibilant tier. This tier contains only the sibilants of the word, and on the tier no two adjacent segments may disagree in anteriority.

(15) Intervocalic voicing (strictly local) and sibilant harmony (tier-based strictly local)

¹²The reader may have noted the similarity to the requirement in Graf (2020) that category features must be inferable from a context of finitely bounded size. There is indeed a principled mathematical connection between this notion of locality and subregular strict locality as discussed here.



Curiously, the most fundamental aspects of MGs show the very same split between strict locality and tier-based strict locality (Graf 2018): Merge is strictly local, and Move is tier-based strictly local. Consider Merge first. In order for a Merge step to be licit, the selector feature of the head h must match the category feature of the selected head s . The maximum distance between h and s depends on the number of positive features each one carries, but it is guaranteed to be finitely bounded for any fixed MG. As exemplified in (16), we only have to consider tree context of finitely bounded size, making Merge strictly local over trees. Movement dependencies, on the other hand, can cross arbitrarily long distances, and thus finitely bounded tree contexts are insufficient. However, for each movement feature f (*wh*, *nom*, *topic*, and so on), one can construct a movement tier that only contains the lexical items carrying f^- and the Move nodes involved in checking these features. On this tier, the long-distance dependency between f^+ and f^- is a strictly local constraint that every f^- must have a Move node as its mother, and every Move node must have exactly one f^- among its daughters.¹³

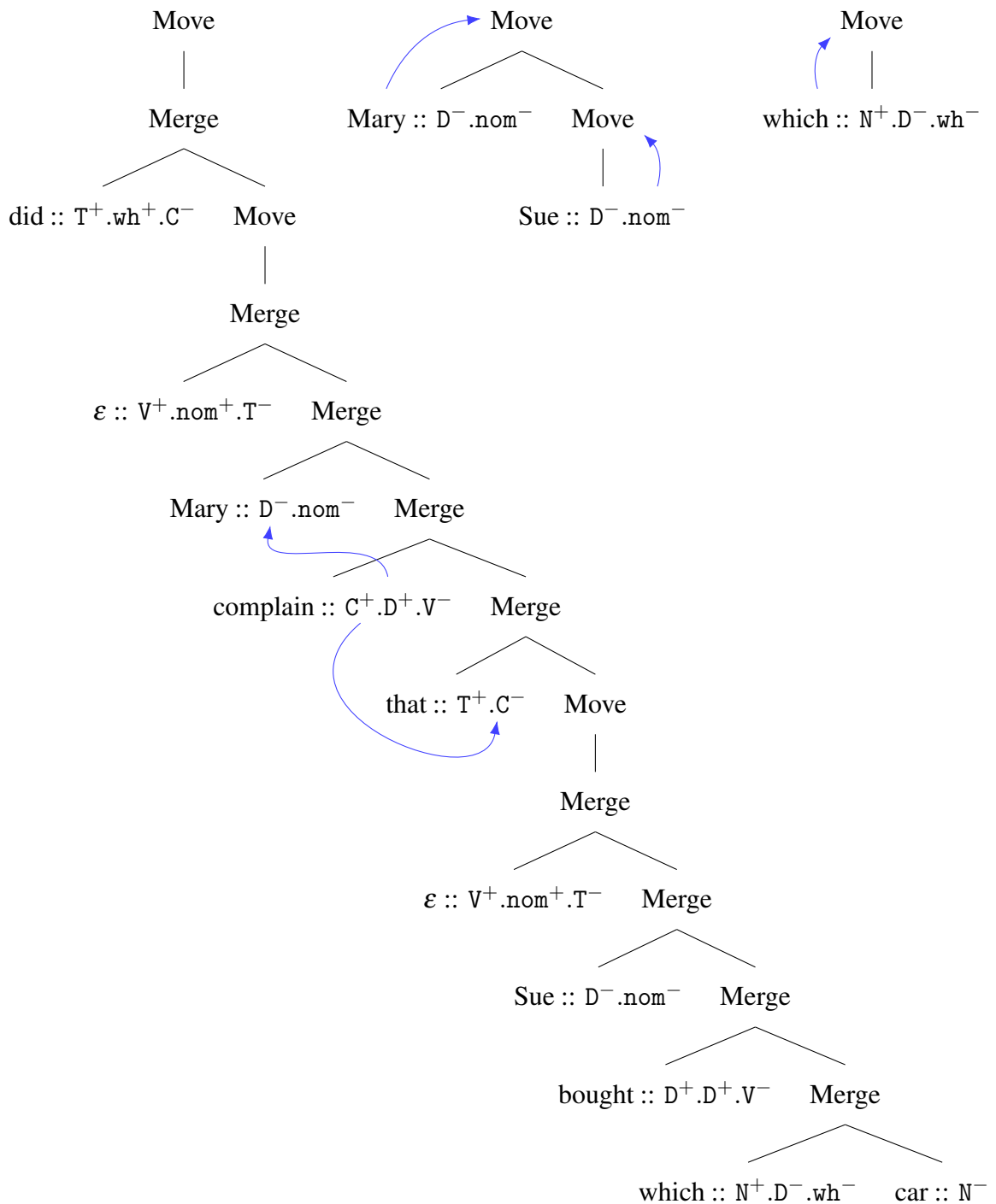
¹³The TSL view of movement only works for a subtype of MGs where licensee features are not linearly ordered. Hence a lexical item like *who* :: $D^- .nom^- .wh^-$ could first undergo *nom*-movement and then *wh*-movement, or the other way round. This subtype of MGs has the same weak and strong generative capacity as standard MGs (a corollary of Graf et al. 2016), but its derivational complexity is slightly lower, which allows it to be analyzed in terms of TSL.

(16)

Derivation

nom-Tier

wh-Tier



It seems, then, that phonology is strictly local and tier-based strictly local over strings, whereas syntax is strictly local (Merge) and tier-based strictly local (Move) over trees. This is a surprising parallel, and many more emerge once one lifts subregular concepts from phonology to syntax.

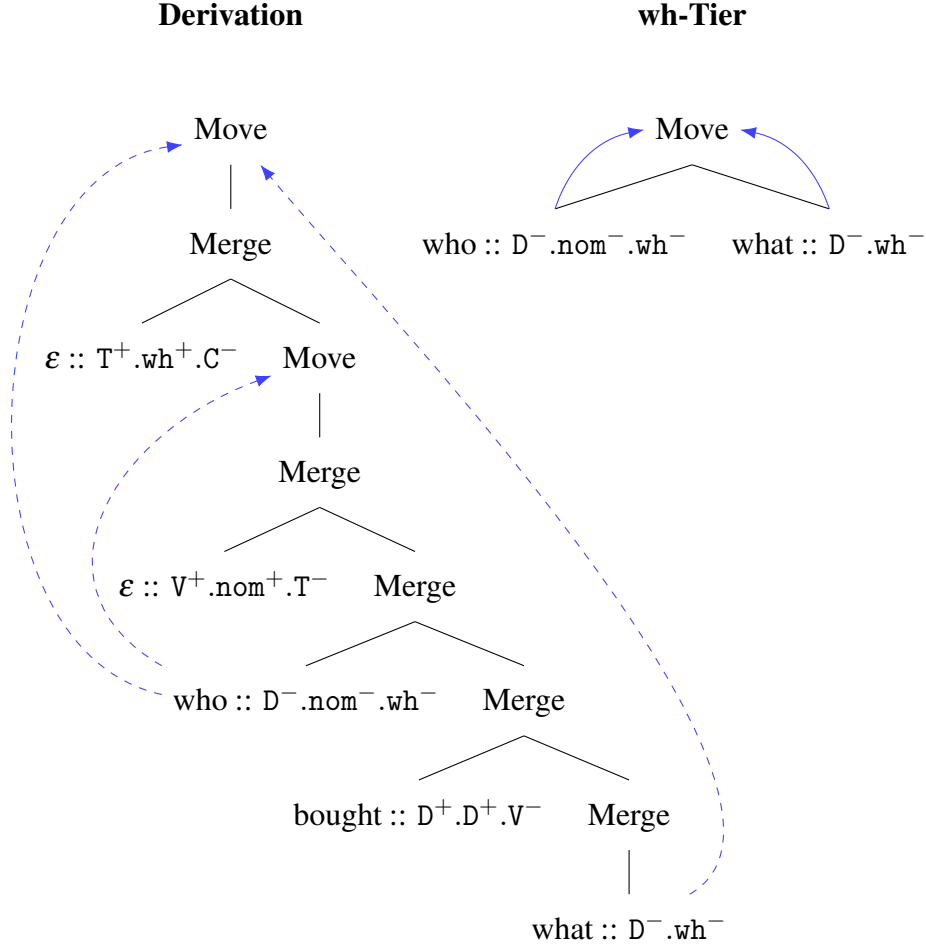
Take the case of Irish *wh*-agreement, where complementizers surface with special morphology if they occur along a *wh*-movement path (McCloskey 2001). From a subregular perspective, this looks very much like unbounded tone plateauing, where no low tone may appear within an interval spanned by two high tones. The high tones correspond to wh^+ and wh^- , and the illicit low tone would be a complementizer that does not show the required morphology. All of this points towards syntax and phonology (and possibly morphology, see Dolatian 2020 and references therein) having a lot more in common than is usually believed. If so, this would have far-reaching repercussions for how we think about these language modules and how we do research, as phonological evidence could suddenly be brought to bear on syntax, and the other way round.

5.3 Multiple *wh*-movement

The tier-based view of movement also provides a novel perspective on a variety of well-known phenomena. Graf (2022) explains how movement tiers explain the existence of island effects, the *that*-trace effect, and the anti-*that*-trace effect, among others. Particularly noteworthy is how easy the tier-based view makes it to relax the SMC to allow for multiple *wh*-movement.

For standard movement, the requirement is that every Move node on an *f*-tier has exactly one daughter with f^- , which is formally equivalent to the SMC of standard MGs. But this is actually the conjunction of two constraints: the Move node must have at least one daughter with f^- , and it must have at most one daughter with f^- . If one drops the second constraint for the *wh*-tier, multiple *wh*-movement is now allowed in the sense that sentences like *who_i what_j t_i bought t_j* can be analyzed as a single Move step checking the licensee features of both *who* and *what*.

(17) A derivation tree with multiple *wh*-movement and its *wh*-tier



This interpretation of multiple wh-movement subverts the received view of movement: standard movement is now a special case of multiple movement that is subject to the additional constraint that the Move node cannot have more than one daughter with the required licensee feature.

Admittedly this view leaves open several key aspects of multiple wh-movement. It does not address how one would compute the c-command relations between these movers, how they should be linearized, and how superiority effects arise. Moreover, it only works for cases where multiple wh-movers target roughly the same position in the clause. Hunter and Frank (2021) point out that these cases are computationally simpler than constructions like *who_i t_i wonders what_j t_j bought t_j*, where the two wh-phrases have final landing sites in distinct clauses. To the extent that such configurations are possible in languages with multiple wh-movement, the tier-based view of movement suggests that they require distinct mechanisms from the usual multiple wh-movement examples like *who what bought*.

One major caveat, though, is that all these considerations are moot if the number of wh-movers is finitely bounded. In this case, even standard MGs with the SMC furnish a variety of accounts for multiple wh-movement, from abstract MSO-solutions to blunt coding tricks like splitting the

feature *wh* into subtypes *wh*1, *wh*2, and so on, to get around the SMC. Multiple *wh*-movement thus provides a testbed for three distinct views of movement: if the number of multiple *wh*-movers is finitely bounded, standard MGs with the SMC will do just fine; if the number is unbounded but all movers target roughly the same position in the clause, the SMC should be relaxed along the lines suggested by the tier-based view; and if the number is both unbounded and all movers can target positions in different clauses, a completely new MG treatment will be needed. The data currently available in the literature provides no clear-cut answer. Examples of multiple *wh*-movement rarely contain more than three *wh*-phrases, and often there are additional restrictions, e.g. that the morphological cases of the *wh*-movers must be pairwise distinct. A language that allows multiple *wh*-movement but requires every mover to have a distinct case necessarily establishes a finite upper bound as there are only finitely many distinct cases. More data is needed before a consensus can emerge as to whether the number of *wh*-movers should be taken to be unbounded.

Overall, then, the computational view of multiple *wh*-movement raises several issues that are rarely the focus of the linguistic literature on this phenomenon: To what extent is the number of *wh*-movers truly unbounded? Are there principled upper bounds, e.g. because multiple *wh*-movement does not allow for two movers that are identical with respect to case, gender, and animacy? If the number of movers is indeed unbounded, are there cases where an unbounded number of *wh*-movers target positions in completely different clauses, akin to the configurations identified in Hunter and Frank (2021)? And if not, are there nonetheless problems with the idea espoused in the tier-based perspective that *wh*-phrases target the same landing site, with the *c*-command relations between them inferred in some other manner? Even though they are prompted by computational considerations, these are profoundly linguistic questions.

5.4 Cliffs notes

Subregular syntax is still a young enterprise, and a lot of work remains to be done. The goal is to identify weaker classes than MSO that are still sufficiently powerful for syntax. Due to the equivalence of features and MSO-constraints, the issue has to be approached from two directions. Subregular notions of locality can constrain the feature system by requiring Merge features to be locally inferable — even though the features are already stored in the lexicon, the nature of the feature system has to be such that it cannot implicitly encode non-local information. At the level of constraints, it seems that strictly local and tier-based strictly local dependencies, which play a major role in subregular phonology, also take center stage in syntax. This suggests that phonology and syntax have a lot more in common than is commonly assumed. Strictly local dependencies apply within structural domains of finitely bounded size, and tier-based strictly local dependencies are strictly local dependencies over a tier structure. In the case of syntax, these are tree tiers, which

provide a novel perspective on long-distance dependencies such as movement. It is unclear if all aspects of syntax are at most tier-based strictly local (Graf and De Santo 2019), but there is plenty of evidence that a much smaller class than MSO is sufficient for syntactic constraints. Because subregular complexity is a much more fine-grained than more widely used notions of complexity in formal language theory, it is also much more sensitive to subtle details of the linguistic data, e.g. with respect to multiple *wh*-movement. This means that syntacticians can play a much more prominent role in determining the subregular bounds of syntax, and how those line up with other aspects of language.

6 MGs as a research tool

The previous sections revolved around research where MGs take center stage, but there is also a rich body of work where MGs play a less prominent yet equally important role: whenever a computationally minded project seeks to incorporate insights from Minimalism, MGs are an easy way to do so.

For example, Yun et al. (2014) present a computational model of relative clause processing in East Asian languages. The formal core of the paper draws from probability theory and information theory, but the analysis of relative clauses is taken from Minimalism. In order to combine these two approaches, one needs I) a rigorously specified grammar that can express Minimalist analyses, and II) a reliable method for calculating probabilities with said grammar. This is exactly what MGs excel at. They provide a very direct implementation of Minimalist ideas, do not leave any parts of the formalism underspecified, and can be made probabilistic by either converting them to MCFGs with probabilities for each rewrite rule, or by defining probability distributions directly over the syntactic derivations (Hunter and Dyer 2013). Without MGs, this work would have been a lot harder, if not impossible.

Another model of sentence processing has been built on the basis of existing MG parsing techniques. Stabler (2011b, 2013), extending earlier work by Mainguy (2010), presented a new top-down parser for MGs. Since we usually think of Minimalist structure building in a bottom-up fashion, it might be surprising that parsing can be done in a top-down fashion. But recall from Sec. 3.2 that MGs are, in a loose sense, CFGs with a more complex linearization procedure, and CFGs are top-down rewriting devices. Hence one can take standard top-down parsing techniques for CFGs and adapt them to MGs by keeping track of how movement affects linearization, which is exactly what Stabler did.¹⁴ Kobele et al. (2013) then took this MG parser and combined it with

¹⁴For an alternative approach that makes top-down structure building a core part of MGs themselves, see Chesi (2021) on expectation-based MGs. This version of MGs also uses a different feature system and makes Agree a core operation of the grammar.

a specific way of measuring its memory usage during the structure building process. Curiously, memory usage lines up with processing difficulty in a number of constructions. For instance, it is well-known that crossing dependencies are easier to parse than nested dependencies (Bach et al. 1986), even though the latter are simpler in terms of formal language theory. But nested dependencies increase the MG parser’s memory usage more than crossing dependencies do. Under the plausible assumption that higher memory usage makes sentence processing harder for humans, this explains why crossing dependencies are easier than nested dependencies.

This approach has since then be applied to many other phenomena, including relative clause processing, attachment ambiguities, heavy NP shift, quantifier scope, gradability, and structural priming (see Gerth 2015, Graf et al. 2017, De Santo 2020, Pasternak and Graf 2021, and references therein). The approach is very accessible and does not require deep familiarity with MGs. The formalism’s role is once again to provide an easy way to take Minimalist analyses and combine them with a computational model, in this case top-down parsing. It is only because of all the mathematical groundwork discussed in Sec. 3 and 4 that we now understand MGs enough to effectively employ them in this approachable, “math-light” manner.

In the future, MGs may allow Minimalist ideas to spread even deeper into neighboring disciplines. Torr et al. (2019) developed the first wide-coverage parser for MGs that is suitable for use in practical applications with a need for rich structural descriptions, e.g. machine translation. The parser builds on a technique called **supertagging**, which has previously been used to great success with TAG and CCG. In a highly lexicalized formalism like MGs, parsing is much easier if each word is already annotated with its feature string — once the feature strings are known, there are far fewer options for how the words should be combined via Merge and Move. Supertagging decomposes parsing into a two-step procedure: feature assignment, and structure building. As the first step, each word in the input is automatically assigned a feature string, usually by a neural network or some other probabilistic device. Crucially, no structural information is used at this point, all decisions are made based only on what the input string looks like. Once all words have been annotated with features, a very fast, probabilistic parsing technique (A^* parsing) quickly computes the most likely tree structure. By design, supertagging does not always find the right parse because the initial feature annotation step only uses string-based heuristics, which can easily result in errors. This is the price one has to pay for the greatly increased speed relative to standard MG parsing techniques (Fowlie and Koller 2017). The accuracy of this supertagging parser is good enough for practical applications, many of which have a greater need for fast parsing than for perfectly accurate parsing.

Not only is this a unique opportunity for Minimalist syntax to raise its profile in other fields, it is also a valuable direction to take from a theoretical perspective: In general, science has benefited greatly from the interplay of theory and application, and language technology may reveal areas

where Minimalism is currently falling short. And more specifically, supertagging is opening up a turf for Minimalism that so far has been dominated by CCG — if one of the two turns out to simply be better-suited to some real-world problems than the other, that is a valuable insight.

Minimalism’s main asset in comparison to other approaches is, arguably, its wide empirical coverage, with numerous analyses for many understudied languages. There is growing awareness that the majority of languages do not have the resources (word lists, corpora, online message boards, and so on) that language technology now relies on in the age of Big Data. One solution to this is a more universal, less language specific framework, as has been advocated by Bender et al. (2002). Minimalist syntax, much more so than other syntactic theories, is built around a large common core shared by all languages, making it the perfect champion of such an approach to language technology.

This, too, is a project that linguists can contribute to in a very concrete manner. The supertagging parser for MGs would not have been possible without the work of Torr (2017), who constructed an English MG corpus from a significant chunk of the Penn treebank. This corpus is the basis for the probabilities used by the supertagger and the A*-parser of Torr et al. (2019). But this MG corpus is still just a starting point; ideally, there would be a plethora of large MG corpora for a variety of languages. This may not be feasible given the required (wo)man power. But even very small corpora have a measurable effect on a system’s performance when they are combined with linguistic insights (Srivastava et al. 2020). There is a lot to be gained from exploring ways to convert syntacticians’ rich body of knowledge into resources that support language technology. While some Minimalists doubt the scientific value of corpus work, this is a case where comparatively little effort could have a tremendous payoff for the whole community, and society at large.

7 Conclusion

Since their inception over 25 years ago, MGs have been the driving force in combining Minimalism and computation. In doing so, they have assumed two very different, yet closely related roles. One is as a mathematical window into Minimalism, granting us deeper understanding of its machinery and thus the claims it makes about language. Examples of this are weak generative capacity arguments for copy movement, the central role of derivations, the expressive equivalence of features and constraints, and the computational parallels between phonology and syntax. The other role is that of a computational wrapper around Minimalism, which makes syntactic proposals easier to incorporate into computational research. This has allowed for several new processing models informed by Minimalist analyses, and with the arrival of a robust wide-coverage parser for MGs, many new applications are suddenly within reach.

Both types of MG work have traditionally required a high degree of mathematical expertise

and thus have seen very little direct involvement from Minimalists. But it seems that a tipping point has been reached in the last few years, with MGs now more approachable and linguistically faithful than ever before, but also in greater need of direct support from linguists. Be it the use of MGs in sentence processing, the design of Minimalist corpora, or the exploration of phenomena that push the limits of subregular complexity, there are now many areas where computational and theoretical linguists can fruitfully collaborate without any major mathematical stumbling blocks along the way.

References

- Adger, David. 2003. *Core syntax: A Minimalist approach*. Oxford: Oxford University Press.
- Adger, David. 2010. A Minimalist theory of feature structure. In *Features: Perspectives on a key notion in linguistics*, ed. Anna Kibort and Greville G. Corbett, 185–218. Oxford: Oxford University Press.
- Amblard, Maxime, Alain Lecomte, and Christian Retoré. 2010. Categorical minimalist grammars: From generative syntax to logical forms. *Linguistic Analysis* 36:273–308.
- Applegate, Richard B. 1972. *Ineseño Chumash grammar*. Doctoral Dissertation, University of California, Berkeley.
- Bach, Emmon, Colin Brown, and William Marslen-Wilson. 1986. Crossed and nested dependencies in German and Dutch: A psycholinguistic study. *Language and Cognitive Processes* 1:249–262.
- Bender, Emily M, Dan Flickinger, and Stephan Oepen. 2002. The grammar matrix: An open-source starter-kit for the rapid development of cross-linguistically consistent broad-coverage precision grammars. In *Proceedings of the 2002 Workshop on Grammar Engineering and Evaluation - Volume 15*, COLING-GEE '02, 1–7. Association for Computational Linguistics.
- Bhatt, Rajesh, and Aravind Joshi. 2004. Semilinearity is a syntactic invariant. A reply to Kracht and Michaelis 1997. *Linguistic Inquiry* 35:683–692.
- Borer, Hagit. 1984. *Parametric syntax: Case studies in semitic and romance languages*. Dordrecht: Foris.
- Chandlee, Jane. 2014. *Strictly local phonological processes*. Doctoral Dissertation, University of Delaware. URL <http://udspace.udel.edu/handle/19716/13374>.

- Chesi, Cristiano. 2021. Expectation-based Minimalist grammars. URL <https://lingbuzz.net/lingbuzz/006135>, manuscript, IUSS Pavia.
- Chomsky, Noam. 1956. Three models for the description of language. *IRE Transactions on Information Theory* 2:113–124.
- Chomsky, Noam. 1957. *Syntactic structures*. The Hague: Mouton.
- Chomsky, Noam. 1959. On certain formal properties of grammars. *Information and Control* 2:137–167. URL [https://doi.org/10.1016/S0019-9958\(59\)90362-6](https://doi.org/10.1016/S0019-9958(59)90362-6).
- Chomsky, Noam. 1965. *Aspects of the theory of syntax*. Cambridge, MA: MIT Press.
- Chomsky, Noam. 1986. *Knowledge of language: Its nature, origin, and use*. New York: Praeger.
- Chomsky, Noam. 1990. On formalization and formal linguistics. *Natural Language and Linguistic Theory* 8:143–147.
- Chomsky, Noam. 1995. *The Minimalist Program*. Cambridge, MA: MIT Press.
- Chomsky, Noam. 1998. *Minimalist inquiries: The framework*, volume 15 of *MIT Occasional Papers in Linguistics*. Cambridge, MA: MIT Press.
- Chomsky, Noam. 2001. Derivation by phase. In *Ken Hale: A life in language*, ed. Michael J. Kenstowicz, 1–52. Cambridge, MA: MIT Press.
- Chomsky, Noam. 2008. On phases. In *Foundational issues in linguistic theory: Essay in honor of Jean-Roger Vergnaud*, ed. Robert Freidin, Carlos P. Otero, and Maria Luisa Zubizarreta, 133–166. Cambridge, MA: MIT Press.
- Chomsky, Noam. 2013. Problems of projection. *Lingua* 130:33–49.
- Culy, Christopher. 1985. The complexity of the vocabulary of Bambara. *Linguistics and Philosophy* 8:345–351. URL https://doi.org/10.1007/978-94-009-3401-6_14.
- De Santo, Aniello. 2020. *Structure and memory: A computational model of storage, gradience, and priming*. Doctoral Dissertation, Stony Brook University.
- De Santo, Aniello, and Thomas Graf. 2019. Structure sensitive tier projection: Applications and formal properties. In *Formal Grammar*, ed. Raffaella Bernardi, Gregory Kobele, and Sylvain Pogodalla, 35–50. Berlin, Heidelberg: Springer.

- Dolatian, Hossep. 2020. *Computational locality of cyclic phonology in Armenian*. Doctoral Dissertation, Stony Brook University.
- Ermolaeva, Marina. 2018. Morphological agreement in Minimalist grammars. In *Proceedings of Formal Grammar 2017*, ed. Annie Foret, Reinhard Muskens, and Sylvain Pogodalla, 20–36. Berlin: Springer.
- Ermolaeva, Marina, and Gregory M. Kobele. 2022. Agree as information transmission over dependencies. *Syntax* 25:466–507.
- Fong, Sandiway, and Jason Ginsburg. 2012. Computation with doubling constituents: Pronouns and antecedents in phase theory. In *Towards a biolinguistic understanding of grammar*, ed. Anna Maria Di Sciullo, 303–338. Amsterdam: John Benjamins.
- Fowlie, Meaghan. 2013. Order and optionality: Minimalist grammars with adjunction. In *Proceedings of the 13th Meeting on the Mathematics of Language (MoL 13)*, ed. András Kornai and Marco Kuhlmann, 12–20.
- Fowlie, Meaghan, and Alexander Koller. 2017. Parsing minimalist languages with interpreted regular tree grammars. In *Proceedings of the 13th International Workshop on Tree Adjoining Grammars and Related Formalisms*, 11–20.
- Frank, Robert. 2004. *Phrase structure composition and syntactic dependencies*. Boston: MIT Press.
- Frey, Werner, and Hans-Martin Gärtner. 2002. On the treatment of scrambling and adjunction in Minimalist grammars. In *Proceedings of the Conference on Formal Grammar*, ed. Gerhard Jäger, Paola Monachesi, Gerald Penn, and Shuly Wintner, 41–52.
- Gärtner, Hans-Martin, and Jens Michaelis. 2007. Some remarks on locality conditions and Minimalist Grammars. In *Interfaces + recursion = language? Chomsky's minimalism and the view from syntax-semantics*, ed. Uli Sauerland and Hans-Martin Gärtner, 161–196. Berlin: Mouton de Gruyter.
- Gärtner, Hans-Martin, and Jens Michaelis. 2010. On the treatment of multiple-wh-interrogatives in Minimalist grammars. In *Language and logos*, ed. Thomas Hanneforth and Gisbert Fanselow, 339–366. Berlin: Akademie Verlag.
- Gazdar, Gerald, Ewan Klein, Geoffrey K. Pullum, and Ivan A. Sag. 1985. *Generalized phrase structure grammar*. Oxford: Blackwell.

- Gerth, Sabrina. 2015. *Memory limitations in sentence comprehension. a structure-based complexity metric of processing difficulty*. Doctoral Dissertation, University of Potsdam.
- Goldsmith, John. 1976. *Autosegmental phonology*. Doctoral Dissertation, MIT.
- Graf, Thomas. 2011. Closure properties of Minimalist derivation tree languages. In *LACL 2011*, ed. Sylvain Pogodalla and Jean-Philippe Prost, volume 6736 of *Lecture Notes in Artificial Intelligence*, 96–111. Heidelberg: Springer. URL https://dx.doi.org/10.1007/978-3-642-22221-4_7.
- Graf, Thomas. 2012a. Movement-generalized Minimalist grammars. In *LACL 2012*, ed. Denis Béchet and Alexander J. Dikovsky, volume 7351 of *Lecture Notes in Computer Science*, 58–73. URL http://dx.doi.org/10.1007/978-3-642-31262-5_4.
- Graf, Thomas. 2012b. Tree adjunction as Minimalist lowering. In *Proceedings of the 11th International Workshop on Tree Adjoining Grammars and Related Formalisms (TAG+11)*, 19–27. URL http://www.aclweb.org/old_anthology/W/W12/W12-4603.pdf.
- Graf, Thomas. 2013. *Local and transderivational constraints in syntax and semantics*. Doctoral Dissertation, UCLA. URL <http://thomasgraf.net/doc/papers/Graf13Thesis.pdf>.
- Graf, Thomas. 2014a. Late merge as lowering movement in Minimalist grammars. In *LACL 2014*, ed. Nicholas Asher and Sergei Soloviev, volume 8535 of *Lecture Notes in Computer Science*, 107–121. Heidelberg: Springer. URL https://doi.org/10.1007/978-3-662-43742-1_9.
- Graf, Thomas. 2014b. Models of adjunction in Minimalist grammars. In *Formal Grammar 2014*, ed. Glynn Morrill, Reinhard Muskens, Rainer Osswald, and Frank Richter, volume 8612 of *Lecture Notes in Computer Science*, 52–68. Heidelberg: Springer. URL https://doi.org/10.1007/978-3-662-44121-3_4.
- Graf, Thomas. 2017. A computational guide to the dichotomy of features and constraints. *Glossa* 2:1–36. URL <https://dx.doi.org/10.5334/gjgl.212>.
- Graf, Thomas. 2018. Why movement comes for free once you have adjunction. In *Proceedings of CLS 53*, ed. Daniel Edmiston, Marina Ermolaeva, Emre Hakgüder, Jackie Lai, Kathryn Montemurro, Brandon Rhodes, Amara Sankhagowit, and Miachel Tabatowski, 117–136.
- Graf, Thomas. 2020. Curbing feature coding: Strictly local feature assignment. In *Proceedings of the Society for Computation in Linguistics (SCiL) 2020*, 362–371.
- Graf, Thomas. 2022. Typological implications of tier-based strictly local movement. In *Proceedings of the Society for Computation in Linguistics (SCiL) 2022*, 184–193.

- Graf, Thomas, Alëna Aksënova, and Aniello De Santo. 2016. A single movement normal form for Minimalist grammars. In *Formal Grammar: 20th and 21st International Conferences, FG 2015, Barcelona, Spain, August 2015, Revised Selected Papers. FG 2016, Bozen, Italy, August 2016*, ed. Annie Foret, Glyn Morrill, Reinhard Muskens, Rainer Osswald, and Sylvain Pogodalla, 200–215. Berlin, Heidelberg: Springer. URL https://doi.org/10.1007/978-3-662-53042-9_12.
- Graf, Thomas, and Aniello De Santo. 2019. Sensing tree automata as a model of syntactic dependencies. In *Proceedings of the 16th Meeting on the Mathematics of Language*, 12–26. Toronto, Canada: Association for Computational Linguistics. URL <https://www.aclweb.org/anthology/W19-5702>.
- Graf, Thomas, and Kalina Kostyszyn. 2021. Multiple wh-movement is not special: The subregular complexity of persistent features in Minimalist grammars. In *Proceedings of the Society for Computation in Linguistics (SCiL) 2021*, 275–285.
- Graf, Thomas, James Monette, and Chong Zhang. 2017. Relative clauses as a benchmark for Minimalist parsing. *Journal of Language Modelling* 5:57–106. URL <http://dx.doi.org/10.15398/jlm.v5i1.157>.
- Haegeman, Liliane. 1994. *Introduction to government and binding theory*. Oxford: Blackwell, 2 edition.
- Harkema, Henk. 2001a. A characterization of Minimalist languages. In *Logical aspects of computational linguistics (LACL'01)*, ed. Philippe de Groote, Glyn Morrill, and Christian Retoré, volume 2099 of *Lecture Notes in Artificial Intelligence*, 193–211. Berlin: Springer.
- Harkema, Henk. 2001b. *Parsing Minimalist languages*. Doctoral Dissertation, University of California.
- Harley, Heidi, and Elizabeth Ritter. 2002. Person and number in pronouns: A feature-geometric analysis. *Language* 78:482–526.
- Heim, Irene. 1998. Anaphora and semantic interpretation: A reinterpretation of Reinhart's approach. In *The interpretive tract*, ed. Uli Sauerland and O. Percus, volume 25 of *MIT Working Papers in Linguistics*, 205–246. Cambridge, MA: MIT Press.
- Heinz, Jeffrey. 2009. On the role of locality in learning stress patterns. *Phonology* 26:303–351. URL <https://doi.org/10.1017/S0952675709990145>.

- Heinz, Jeffrey. 2018. The computational nature of phonological generalizations. In *Phonological typology*, ed. Larry Hyman and Frank Plank, Phonetics and Phonology, chapter 5, 126–195. Mouton De Gruyter.
- Heinz, Jeffrey, Chetan Rawal, and Herbert G. Tanner. 2011. Tier-based strictly local constraints in phonology. In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics*, 58–64. URL <http://www.aclweb.org/anthology/P11-2011>.
- Hunter, Tim. 2015. Deconstructing merge and move to make room for adjunction. *Syntax* 18:266–319.
- Hunter, Tim, and Chris Dyer. 2013. Distributions on Minimalist grammar derivations. In *Proceedings of the 13th meeting on the mathematics of language (mol 13)*, 1–11.
- Hunter, Tim, and Robert Frank. 2021. Comparing methods of tree-construction across mildly context-sensitive formalism. In *Proceedings of the society for computation in linguistics*, volume 4, 355–358.
- Huybregts, Riny. 1984. The weak adequacy of context-free phrase structure grammar. In *Van periferie naar kern*, ed. Ger J. de Haan, Mieke Trommelen, and Wim Zonneveld, 81–99. Dordrecht: Foris.
- Jardine, Adam. 2016. Computationally, tone is different. *Phonology* 33:247–283. URL <https://doi.org/10.1017/S0952675716000129>.
- Joshi, Aravind. 1985. Tree-adjoining grammars: How much context sensitivity is required to provide reasonable structural descriptions? In *Natural language parsing*, ed. David Dowty, Lauri Karttunen, and Arnold Zwicky, 206–250. Cambridge: Cambridge University Press.
- Joshi, Aravind, and Y. Schabes. 1997. Tree-adjoining grammars. In *Handbook of formal languages*, ed. Grzegorz Rosenberg and Arto Salomaa, 69–123. Berlin: Springer.
- Joshi, Aravind, K. Vijay-Shanker, and David Weir. 1991. The convergence of mildly context-sensitive grammar formalisms. In *Foundational issues in natural language processing*, ed. Peter Sells, Stuart M. Shieber, and Thomas Wasow, 31–81. Cambridge, MA: MIT Press.
- Kallmeyer, Laura. 2010. *Parsing beyond context-free grammars*. Berlin: Springer.
- Kaplan, Ronald M., and Martin Kay. 1994. Regular models of phonological rule systems. *Computational Linguistics* 20:331–378. URL <http://www.aclweb.org/anthology/J94-3001.pdf>.

- Kepser, Stephan, and James Rogers. 2011. The equivalence of tree adjoining grammars and monadic linear context-free tree grammars. *Journal of Logic, Language and Information* 20:361–384.
- Kobele, Gregory M. 2006. *Generating copies: An investigation into structural identity in language and grammar*. Doctoral Dissertation, UCLA. URL <https://home.uni-leipzig.de/gkobele/files/unpub/Kobele06GeneratingCopies.pdf>.
- Kobele, Gregory M. 2008. Across-the-board extraction and Minimalist grammars. In *Proceedings of the Ninth International Workshop on Tree Adjoining Grammars and Related Frameworks*, ed. Claire Gardent and Anoop Sarkar, 113–128. ACL. URL <https://aclanthology.org/W08-2315.pdf>.
- Kobele, Gregory M. 2010. Without remnant movement, MGs are context-free. In *MOL 10/11*, ed. Christian Ebert, Gerhard Jäger, and Jens Michaelis, volume 6149 of *Lecture Notes in Computer Science*, 160–173.
- Kobele, Gregory M. 2011. Minimalist tree languages are closed under intersection with recognizable tree languages. In *LACL 2011*, ed. Sylvain Pogodalla and Jean-Philippe Prost, volume 6736 of *Lecture Notes in Artificial Intelligence*, 129–144. URL https://doi.org/10.1007/978-3-642-22221-4_9.
- Kobele, Gregory M. 2012. Idioms and extended transducers. In *Proceedings of the 11th International Workshop on Tree Adjoining Grammars and Related Formalisms (TAG+11)*, 153–161. Paris, France. URL <http://www.aclweb.org/anthology-new/W/W12/W12-4618>.
- Kobele, Gregory M. 2015. LF-copying without LF. *Lingua* 166:236–259.
- Kobele, Gregory M., Sabrina Gerth, and John T. Hale. 2013. Memory resource allocation in top-down Minimalist parsing. In *Formal Grammar: 17th and 18th International Conferences, FG 2012, Opole, Poland, August 2012, Revised Selected Papers, FG 2013, Düsseldorf, Germany, August 2013*, ed. Glyn Morrill and Mark-Jan Nederhof, 32–51. Berlin, Heidelberg: Springer. URL https://doi.org/10.1007/978-3-642-39998-5_3.
- Kobele, Gregory M., and Jens Michaelis. 2005. Two type-0 variants of Minimalist grammars. In *FG-MoL 2005. The 10th conference on Formal Grammar and the 9th Meeting on Mathematics of Language*, 81–93. Edinburgh.
- Kobele, Gregory M., and Jens Michaelis. 2011. Disentangling notions of specifier impenetrability. In *The Mathematics of Language*, ed. Makoto Kanazawa, András Kornia, Marcus Kracht,

- and Hiroyuki Seki, volume 6878 of *Lecture Notes in Artificial Intelligence*, 126–142. Berlin, Heidelberg: Springer. URL https://doi.org/10.1007/978-3-642-23211-4_8.
- Kobele, Gregory M., Christian Retoré, and Sylvain Salvati. 2007. An automata-theoretic approach to Minimalism. In *Model Theoretic Syntax at 10*, ed. James Rogers and Stephan Kepser, 71–80.
- Kuhlmann, Marco, Alexander Koller, and Giorgio Satta. 2015. Lexicalization and generative power in CCG. *Computational Linguistics* 41:215–247.
- Laszakovits, Sabine. 2018. Case theory in Minimalist grammars. In *Proceedings of Formal Grammar 2018*, ed. Annie Foret, Greg Kobele, and Sylvain Pogodalla, 37–61. Berlin: Springer.
- Maclachlan, Anna, and Owen Rambow. 2002. Cross-serial dependencies in Tagalog. In *Proceedings of the Sixth International Workshop on Tree Adjoining Grammar and Related Frameworks (TAG+6)*, 252–258. URL <https://aclanthology.org/W02-2235.pdf>.
- Mainguy, Thomas. 2010. A probabilistic top-down parser for Minimalist grammars. ArXiv:1010.1826v1.
- McCloskey, James. 2001. The morphosyntax of wh-extraction in Irish. *Journal of Linguistics* 37:67–100.
- Michaelis, Jens. 2001. Transforming linear context-free rewriting systems into Minimalist grammars. In *LACL '01: Proceedings of the 4th International Conference on Logical Aspects of Computational Linguistics*, volume 2099 of *Lecture Notes in Artificial Intelligence*, 228–244.
- Michaelis, Jens. 2009. An additional observation on strict derivational minimalism. In *FG-MOL 2005*, ed. James Rogers, 101–111.
- Michaelis, Jens, and Marcus Kracht. 1997. Semilinearity as a syntactic invariant. In *Logical Aspects of Computational Linguistics*, ed. Christian Retoré, volume 1328 of *Lecture Notes in Artificial Intelligence*, 329–345. Springer. URL <http://dx.doi.org/10.1007/BFb0052165>.
- Mohri, Mehryar, and Richard Sproat. 2006. On a common computational fallacy in computational linguistics. In *A man of measure: Festschrift in honour of Fred Karlsson on his 60th birthday*, 432–439. Turku: The Linguistic Association of Finland.
- Mönnich, Uwe. 2006. Grammar morphisms. Ms. University of Tübingen.
- Mönnich, Uwe. 2007. Minimalist syntax, multiple regular tree grammars and direction preserving tree transductions. In *Model Theoretic Syntax at 10*, ed. James Rogers and Stephan Kepser, 83–87.

- Morawietz, Frank. 2003. *Two-step approaches to natural language formalisms*. Berlin: Walter de Gruyter.
- Müller, Gereon. 2010. On deriving CED effects from the PIC. *Linguistic Inquiry* 41:35–82.
- Nevins, Andrew. 2007. The representation of third person and its consequences for person-case effects. *Natural Language and Linguistic Theory* 25:273–313. URL <https://doi.org/10.1007/s11049-006-9017-2>.
- Nevins, Andrew. 2011. Multiple agree with clitics: Person complementarity vs. omnivorous number. *Natural Language and Linguistic Theory* 28:939–971. URL <https://dx.doi.org/10.1007/s11049-006-9017-2>.
- Pasternak, Robert, and Thomas Graf. 2021. Cyclic scope and processing difficulty in a Minimalist parser. *Glossa* 6:1–34.
- Pesetsky, David, and Esther Torrego. 2007. The syntax of valuation and the uninterpretability of features. In *Phrasal and clausal architecture*, ed. Simin Karimi, Samiian Samiian, and Wendy K. Wilkins, 262–294. Amsterdam: Benjamins.
- Peters, Stanley, and Robert W. Ritchie. 1971. On restricting the base component of transformational grammars. *Information and Control* 18:483–501.
- Peters, Stanley, and Robert W. Ritchie. 1973a. Non-filtering and local-filtering transformational grammars. In *Approaches to natural language*, ed. Jaakko Hintikka, J.M.E. Moravcsik, and Patrick Suppes, 180–194. Dordrecht: Reidel.
- Peters, Stanley, and Robert W. Ritchie. 1973b. On the generative power of transformational grammars. *Information Sciences* 6:49–83.
- Pollard, Carl, and Ivan Sag. 1994. *Head-driven phrase structure grammar*. Stanford and Chicago: CSLI and The University of Chicago Press.
- Pullum, Geoffrey K., and Gerald Gazdar. 1982. Natural languages and context free languages. *Linguistics and Philosophy* 4:471–504.
- Radzinski, Daniel. 1991. Chinese number names, tree adjoining languages, and mild context sensitivity. *Computational Linguistics* 17:277–300. URL <http://ucrel.lancs.ac.uk/acl/J/J91/J91-3002.pdf>.
- Reinhart, Tanya. 1983. *Anaphora and semantic interpretation*. Chicago University Press: Croon-Helm.

- Richards, Norvin. 2016. *Contiguity theory*. Cambridge, MA: MIT Press.
- Rogers, James. 1998. *A descriptive approach to language-theoretic complexity*. Stanford: CSLI.
- Salvati, Sylvain. 2011. Minimalist grammars in the light of logic. In *Logic and grammar — essays dedicated to Alain Lecomte on the occasion of his 60th birthday*, ed. Sylvain Pogodalla, Myriam Quatrini, and Christian Retoré, number 6700 in *Lecture Notes in Computer Science*, 81–117. Berlin: Springer.
- Shieber, Stuart M. 1985. Evidence against the context-freeness of natural language. *Linguistics and Philosophy* 8:333–345. URL <http://dx.doi.org/10.1007/BF00630917>.
- Srivastava, Aarohi, Robert Frank, Sarah Widder, and David Chartash. 2020. The role of linguistic features in domain adaptation: TAG parsing of questions. In *Proceedings of the Society for Computation in Linguistics*, volume 3. URL <https://scholarworks.umass.edu/scil/vol3/iss1/41>.
- Stabler, Edward P. 1997. Derivational Minimalism. In *Logical aspects of computational linguistics*, ed. Christian Retoré, volume 1328 of *Lecture Notes in Computer Science*, 68–95. Berlin: Springer. URL <https://doi.org/10.1007/BFb0052152>.
- Stabler, Edward P. 2006. Sideways without copying. In *Formal Grammar '06, Proceedings of the Conference*, ed. Gerald Penn, Giorgio Satta, and Shuly Wintner, 133–146. Stanford: CSLI.
- Stabler, Edward P. 2011a. Computational perspectives on Minimalism. In *Oxford handbook of linguistic Minimalism*, ed. Cedric Boeckx, 617–643. Oxford: Oxford University Press.
- Stabler, Edward P. 2011b. Top-down recognizers for MCFGs and MGs. In *Proceedings of the 2nd Workshop on Cognitive Modeling and Computational Linguistics*, 39–48.
- Stabler, Edward P. 2013. Two models of minimalist, incremental syntactic analysis. *Topics in Cognitive Science* 5:611–633. URL <https://dx.doi.org/10.1111/tops.12031>.
- Stabler, Edward P., and Edward Keenan. 2003. Structural similarity within and among languages. *Theoretical Computer Science* 293:345–363.
- Stanojević, Miloš, and Edward Stabler. 2018. A sound and complete left-corner parser for Minimalist grammars. In *Proceedings of the 8th Workshop on Cognitive Aspects of Computational Language Learning and Processing*, 65–74.
- Steedman, Mark, and Jason Baldridge. 2003. Combinatory categorial grammar. Unpublished tutorial paper.

- Svenonius, Peter. 2019. Syntactic features. In *Oxford research encyclopedia of linguistics*. Oxford: Oxford University Press.
- Svenonius, Peter. 2020. A span is a thing: A span-based theory of words. In *NELS 50: Proceedings of the Fiftieth Annual Meeting of the North East Linguistic Society*, ed. Mariam Asatryan, Yixiao Song, and Ayana Whitmal, 193–206.
- Torr, John. 2017. Autobank: a semi-automatic annotation tool for developing deep Minimalist grammar treebanks. In *Proceedings of the Demonstrations at the 15th Conference of the European Chapter of the Association for Computational Linguistics*, 81–86.
- Torr, John, and Edward P. Stabler. 2016. Coordination in Minimalist grammars: Excorporation and across the board (head) movement. In *Proceedings of the 12th International Workshop on Tree Adjoining Grammars and Related Formalisms (TAG+12)*, 1–17. Düsseldorf, Germany. URL <https://www.aclweb.org/anthology/W16-3301>.
- Torr, John, Miloš Stanojević, Mark Steedman, and Shay Cohen. 2019. Wide-coverage neural A* parsing for Minimalist grammars. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Florence, Italy: Association for Computational Linguistics.
- Vijay-Shanker, K., and David J. Weir. 1994. The equivalence of four extensions of context-free grammars. *Mathematical Systems Theory* 27:511–545.
- Yang, Charles D. 2002. *Knowledge and learning in natural language*. Oxford: Oxford University Press.
- Yun, Jiwon, Zhong Chen, Tim Hunter, John Whitman, and John Hale. 2014. Uncertainty in processing relative clauses across East Asian languages. *Journal of East Asian Linguistics* 1–36.